

**KISSANSURE: AI-BASED CROP INSURANCE
SYSTEM FOR FARMERS**



SUBMITTED BY

Ushba Zahid

2022-ag-7710

Izzah Khursheed

2022-ag-7726

ADVISED BY

Dr. Syed Mushhad Mustuzhar Gilani

A FINAL YEAR PROJECT SUBMITTED FOR THE COURSE

CS-610 FINAL YEAR PROJECT-II 4(0-4)

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF SCIENCES

UNIVERSITY OF AGRICULTURE FAISALABAD

DECLARATION

We hereby declare that the contents of the Final Year Project “**KissanSure: AI-Based Crop Insurance System for Farmers**” are of our own work and no part has been copied from any published source (except the references). We further declare that this work has not been submitted for award of any other diploma/degree. The university may take action if the information provided is found false at any stage. In case of any default the scholar will be proceeded against as per UAF policy.

Ushba Zahid
Izzah Khursheed

Project Evaluation Matrix (Total: 80 Marks)

Category	Obtained Marks	Max Marks
Project Documentation		15
Originality & Code Quality		10
Features Developed		20
User Interface (UI/UX)		10
Portability & Deployment		10
Technical Knowledge (Viva)		15

CERTIFICATE

To,
The Controller of Examinations,
University of Agriculture,
Faisalabad.

The committee certify that **Ushba Zahid 2022-ag-7710 & Izzah Khursheed 2022-ag-7726** has successfully completed her project in partial fulfillment of requirements for the course **CS-610 Final Year Project 4(0-4)** Computer Science under our guidance and supervision.

Advisor: Dr. Syed Mushhad Mustuzhar Gilani

Evaluation Committee Member:

Chairman,
Dr. Saqib Ali
Department of Computer Science

PROJECT'S REPOSITORY QR CODE

KissanSure



ACKNOWLEDGEMENT

In the name of Allah, the Most Gracious, the Most Merciful.

All praise and glory be to Allah Almighty, whose endless blessings, mercy, and guidance made this work possible. It is by His will alone that this project came into existence, and it is to Him that all gratitude ultimately belongs. Our deepest and most heartfelt gratitude goes to our parents, whose unconditional love, endless prayers, and unwavering support have been our greatest strength throughout this journey. Every milestone we reach is a reflection of their sacrifices, and no words can ever fully express what they mean to us. We are sincerely grateful to the Department of Computer Science, University of Agriculture Faisalabad, for providing us with the knowledge, resources, and environment that made this work possible. We extend our special thanks to our supervisor, Dr. Syed Mushhad Mustuzhar Gilani, for his invaluable guidance, patience, and encouragement at every stage of this project. His insightful feedback and constructive challenges shaped this work into what it is today. It is through his supervision that this project found its direction and purpose. We also acknowledge our fellow students and colleagues whose thoughtful discussions and productive feedback contributed new perspectives to this work. For any shortcomings or errors that remain, we take full responsibility.

ABSTRACT

Agriculture is the mainstay of the country's economy and engages a large number of families in the rural areas in farming along with making significant contributions to the GDP of the country. Although agriculture forms such an important part of the national economy, the Pakistani farmers, especially those in Punjab province, are exposed to uncertainties like flood, hailstorm, pest attacks, and drought that affect their financial status adversely. KissanSure is an AI-based crop insurance management platform that offers solutions to the prevailing problems by means of automation and ease of use. The online system provides farmers with a number of features to help them manage their insurance policies for wheat and rice. They can register themselves, purchase crop insurance policy, post images for damage claims, and get the damage assessments using AI technology. This system is developed in PHP for web applications, Python (FastAPI) for artificial intelligence models, MySQL database, and PyTorch deep learning framework. RAD (Rapid Application Development) software engineering approach was followed during the development process to provide a rapid delivery of software products. Testing outcomes show that the proposed system satisfies all the requirements of the project, such as farmer registration, purchase of insurance plan, verification of payment, AI-based claim evaluation, decision-making by admin, and making payouts. The AI models have shown acceptable accuracy in classifying rice and wheat crop images as either healthy or damaged. The system analyzes the uploaded crop images using separate rice and wheat models and then provides an average prediction result based on the crop condition detected. Thus, KissanSure can be considered an important step towards digitalization of agricultural insurance in Pakistan.

Table of Contents

Chapter 1 – INTRODUCTION.....	1
1.1 Background.....	1
1.2 Description.....	1
1.3 Problem Statement.....	2
1.4 Scope.....	2
1.5 Objectives	3
1.6 Feasibility.....	4
1.7 Requirements.....	5
1.7.1 Functional Requirements	5
1.7.2 Non- Functional Requirements	9
1.7.3 Hardware Requirements.....	10
1.7.4 Software Requirements.....	10
1.8 Stakeholders	11
Chapter 2 – MATERIALS & METHODS.....	13
2.1 Process Model.....	13
2.2 Tools & Technologies.....	14
2.3 Design	15
2.3.1 Use Case Diagrams	15
2.3.2 Usage Scenario.....	17
2.3.3 Sequence Diagram	23
2.3.4 Class Diagram.....	32
2.3.5 Data Flow Diagram.....	32
2.3.6 ER Diagram	36
2.3.7 Database Model	37
2.3.8 Architecture.....	38
Chapter 3 - RESULTS & DISCUSSION.....	39
3.1 Testing.....	39
3.2 Test Cases	39
3.3 Conclusion	45
Chapter 4 - USER MANUAL.....	47
4.1 Farmer Portal	47

4.1.1 Home Page	47
4.1.2 Farmer Registration	49
4.1.3 Farmer Login	49
4.1.4 Password Recovery	50
4.1.5 Insurance Plans	51
4.1.6 Insurance Application Form	51
4.1.7 Premium Invoice.....	52
4.1.8 Submit Payment Proof.....	53
4.1.9 My Applications.....	54
4.1.10 File a Damage Claim	55
4.1.11 AI Analysis Result	56
4.1.12 My Claims.....	56
4.1.13 AI Damage Report	57
4.1.14 Payout Receipt	58
4.1.15 Payment History.....	58
4.1.16 Farmer Profile	59
4.1.17 Notifications.....	59
References.....	61

List of Figures

Figure 1.1 Stakeholders	12
Figure 2.1 RAD Model	13
Figure 2.2 Use Case Diagram	16
Figure 2.3 Farmer Registration Sequence Diagram.....	23
Figure 2.4 Farmer Login Sequence Diagram.....	24
Figure 2.5 Buy Insurance Plan Sequence Diagram	25
Figure 2.6 Submit Payment Proof Sequence Diagram	26
Figure 2.7 Admin Verify Payment & Activate Plan Sequence Diagram	27
Figure 2.8 Submit Damage Claim Sequence Diagram	28
Figure 2.9 AI Damage Analysis Sequence Diagram	29
Figure 2.10 Admin Review & Decide Claim Sequence Diagram	30
Figure 2.11 Process Claim Payout Sequence Diagram.....	31
Figure 2.12 Class Diagram	32
Figure 2.13 Context Diagram	33
Figure 2.14 Level 0 DFD.....	33
Figure 2.15 Level 1 DFD	34
Figure 2.16 Level 2 DFD.....	34
Figure 2.17 Entity Relationship Diagram	36
Figure 2.18 Database Model	37
Figure 2.19 Applications's Architecture	38
Figure 4.1 Navigation Bar.....	47
Figure 4.2 Hero Carousel.....	48
Figure 4.3 Contact Us Section	48
Figure 4.4 Farmer Registration Page	49
Figure 4.5 Farmer Login	50
Figure 4.6 Password Recovery Pages	50
Figure 4.7 Insurance Plans Page	51
Figure 4.8 Insurance Application Form	52
Figure 4.9 Premium Invoice Page.....	53

Figure 4.10 Submit Payment Proof Page	54
Figure 4.11 My Applications Page	55
Figure 4.12 File a Damage Claim Page	56
Figure 4.13 AI Analysis Results Display.....	56
Figure 4.14 My Claims Page	57
Figure 4.15 AI Damage Report Page.....	58
Figure 4.16 Payout Receipt Page	58
Figure 4.17 Payment History Page	59
Figure 4.18 Farmer Profile Page.....	59
Figure 4.19 Notification Dropdown.....	60

List of Tables

Table 2.1: Comparison of RAD Phases and Their Application in the Project	14
Table 2.2: Farmer Registration	17
Table 2.3: Farmer Login	17
Table 2.4: Buy Insurance Plan	18
Table 2.5: Submit Payment Proof.....	19
Table 2.6: Admin Verify Payment & Activate Plan	19
Table 2.7: Submit Damage Claim.....	20
Table 2.8: AI Damage Analysis.....	21
Table 2.9: Admin Review & Decide Claim.....	22
Table 2.10: Process Claim Payout	22
Table 2.11: Data Dictionary.....	35
Table 3.1: Farmer Registration	39
Table 3.2: Farmer Login	40
Table 3.3: Buy Insurance Plan	40
Table 3.4: Submit Payment Proof.....	41
Table 3.5: Admin Verify Payment & Activate Plan	42
Table 3.6: Submit Damage Claim.....	43
Table 3.7: AI Damage Analysis.....	43
Table 3.8: Admin Review & Decide Claim.....	44
Table 3.9: Process Claim Payout	45

Chapter 1 - INTRODUCTION

1.1 Background:

Agriculture is central to Pakistan's economy, contributing ~23% to GDP [1], employing ~36% of the labor force [2], and sustaining over 60% of the rural population. Major crops, wheat, rice, cotton, maize, and sugarcane, serve as both the primary food source and a key export commodity. Yet Pakistani farmers face severe recurring threats. Natural disasters, pest infestations, and disease outbreaks regularly destroy entire harvests. The 2022 floods alone submerged over 9.4 million acres of cropland across Sindh and Balochistan [3]. Climate change has made such events more frequent and unpredictable.

Crop insurance is a globally recognized solution to these losses, enabling farmers to recover financially after damage. Countries like India and China have implemented large-scale programs protecting millions of farmers [4]. In Pakistan, however, adoption remains extremely low due to complex paper-based processes, slow manual damage assessments, lack of awareness, and perceived opacity.

Digital technology and AI offer a clear opportunity to address these barriers. Automating the insurance lifecycle and using AI-powered image analysis for objective damage assessment can make the process faster, fairer, and more accessible. KissanSure: AI-Based Crop Insurance System for Farmers was developed to bridge exactly this gap, delivering a modern, transparent, and automated crop insurance solution for Pakistani farmers.

1.2 Description:

KissanSure is a web-based crop insurance management system that automates the complete insurance lifecycle, from farmer registration and plan application to AI-powered damage assessment and financial payout, built specifically for the Pakistani agricultural context.

The platform consists of two portals. The **Farmer Portal** allows unauthenticated users to browse available plans. Registered farmers can apply for a plan by submitting land and crop details; the system automatically calculates the insurance premium and generates an invoice. After the farmer uploads payment proof, an admin verifies it and activates the policy. To file a damage claim, the farmer describes the damage, enters the affected area, and uploads exactly six crop images. Each image is hashed client-side using SHA-256 to detect and reject duplicates. A MobileNetV2-based AI model then classifies each image as Damaged or Healthy with a confidence score, supporting wheat and rice crops. Claim eligibility and payout are calculated based on the proportion of damaged images and the policy's coverage and deductible rates. The **Admin Portal** provides a dashboard with key metrics, and tools to manage farmer accounts, insurance plans, payment verification, claim approvals, and payout issuance. Every farmer-affecting action triggers an in-system notification. Insurance plans support two crop types (wheat, rice) across two crop cycles, Kharif (summer) and Rabi (winter).

The system is built on PHP, MySQL, Bootstrap 5, and a Python FastAPI microservice for AI model serving.

Key limitations include dependency on image quality for accurate AI results, limited internet connectivity in rural areas, and the operational requirement of running the FastAPI microservice alongside the PHP application

1.3 Problem Statement:

Pakistani smallholder farmers face severe financial losses from crop damage yet remain largely unprotected due to the following core problems:

- Crop insurance enrollment is paper-based and inaccessible to rural farmers who lack the time, transportation, and literacy to navigate bureaucratic processes
- Lacking digital documentation makes enrolment difficult and often leads to errors
- Physical field inspections required for damage assessment take weeks, by which time the damaged crop has already deteriorated, making accurate evaluation impossible
- Subjective human assessments are prone to inconsistency and fraud, leading to distrust among farmers who do not understand how claims are evaluated or why they are rejected
- Delayed payouts arrive too late to be useful, forcing farmers into debt, distress asset sales, or permanent abandonment of farming
- Small-scale farmers are financially excluded from insurance because the slow and unfair claims process makes it unviable for them

A web-based, AI-integrated platform that automates damage assessment through image analysis and digitalizes the complete insurance lifecycle offers the most practical and scalable solution to bridge this protection gap for Pakistani farmers.

1.4 Scope:

KissanSure is scoped as a web-based platform deployable on a local XAMPP server for the current academic demonstration, with the architectural foundations ready for cloud deployment. The scope defines all work that needs to be accomplished to deliver the specified features and functions of the system.

In Scope:

The scope of current version includes:

- Farmer self-registration, profile management, and password recovery
- Insurance plan browsing for all visitors and plan creation, configuration, and management by administrators
- Insurance application submission with automated premium calculation and printable invoice generation
- Premium payment proof submission and admin verification workflow
- Damage claim filing with six-image photographic evidence upload

- AI-powered crop damage image analysis for wheat and rice using MobileNetV2
- Claim lifecycle tracking with status updates
- Automated payout calculation based on AI results and plan parameters
- Printable and downloadable AI damage assessment reports and payout receipts
- Farmer notification system for key events (policy activation, claim updates, payout issuance)
- Admin dashboard with live KPI metrics and performance charts
- Admin CRUD operations for farmer accounts, insurance plans, applications, and claims
- Payment history and invoice viewing for farmers
- Contact form with email notification to management via EmailJS

Out of Scope:

These are identified as future extensions:

- SMS or email notifications to farmers
- Payment gateway integration for online premium payment
- AI damage analysis for crops other than wheat and rice
- Integration with government databases (NADRA, provincial agriculture departments)
- Multi-language support (Urdu)
- GPS-based field boundary mapping

1.5 Objectives:**Goal:**

The primary goal of KissanSure is to provide Pakistani smallholder farmers with an accessible, transparent, and AI-assisted crop insurance platform that reduces the time and cost of claim processing.

Specific objectives:

1. To develop a responsive farmer-facing web portal supporting the complete insurance lifecycle from registration to claim tracking.
2. To design and implement a farmer registration and authentication system with secure password management.
3. To implement automated premium calculation engine based on expected yield, market crop prices, coverage level, and base premium rate to eliminate manual computation errors and improve transparency.
4. To build an administrative panel that enables insurance staff to manage farmers, plans, applications, and claims without technical knowledge.
5. To create a digital payment proof submission and admin verification workflow that activates farmer policies upon payment confirmation.
6. To build a damage claim submission interface with six-image evidence upload capability.

7. To integrate a PyTorch-based AI microservice (MobileNetV2) that classifies crop damage images and provides confidence scores for each image.
8. To implement an automated payout calculation formulae based on AI-assessed damage proportion, coverage amount, and deductible rate.
9. To develop a real-time farmer notification system for key events (policy activation, claim status updates, payout issuance).
10. To design an admin dashboard with key performance indicators and visual analytics for portfolio monitoring.
11. To provide farmers with detailed AI assessment reports and downloadable invoices and payout receipts.
12. To ensure that the final system is secure, scalable, and maintainable, capable of supporting additional crops or AI models in the future.
13. Secure all authentication flows using bcrypt password hashing and PHP session management.

1.6 Feasibility:

For the proposed AI-Based Crop Insurance System for Farmers, different types of feasibility analyses were performed to ensure the successful implementation of the project.

1.6.1 Technical Feasibility

The project is technically feasible using freely available technologies. PHP and MySQL provide a mature, well-documented foundation for web development. Bootstrap 5 ensures responsive cross-device compatibility. The AI component uses PyTorch with MobileNetV2, a lightweight, accurate architecture suited for resource-constrained environments. XAMPP serves as the local development server. All tools are zero-cost and the required skills are within the team's competence.

1.6.2 Schedule Feasibility

The project is feasible within the available time duration of the Final Year Project. The system modules such as farmer management, insurance application handling, claim processing, and AI analysis can be completed according to the project timeline. Proper planning and task division help ensure timely completion of the project.

1.6.3 Economic Feasibility

The project is economically feasible because it does not require expensive hardware or paid software resources. The development tools and technologies used, PHP, MySQL, Python, FastAPI, Bootstrap, PyTorch and XAMPP, are open-source and freely available. For deployment, free tiers of hosting options are available. The system can reduce manual work, paperwork, and processing time, which makes it cost-effective in the long run.

1.6.4 Cultural Feasibility

The system uses familiar local terminology (Maund, Kharif/Rabi, PKR) and CNIC-based verification to ensure cultural appropriateness. Its mobile-responsive design supports access from basic smartphones. By eliminating paper-based processes, it promotes digital transformation in agriculture and reduces environmental footprint.

1.6.5 Legal/Ethical Feasibility

The system protects user privacy by securing all personal and financial data with restricted database access and bcrypt password hashing. Farmers can only access their own data; admins have administrative access only.

The AI-based damage assessment removes human bias by evaluating every claim objectively under the same criteria. Per-image confidence scores are displayed to both farmer and admin, ensuring a fair and transparent process.

1.6.6 Resource Feasibility

The required resources for the project are available, including laptops, internet access, development tools, hosting platforms, and database systems. For AI, the MobileNetV2 architecture is available. Two custom models (wheat and rice) will be trained on crop-specific datasets via Google Colab, for which datasets are available on Kaggle and Mendeley.

Human resources are also sufficient because the project can be managed and developed effectively by the development team.

1.6.7 Operational Feasibility

The system is designed to be operated by a trained insurance administrator with basic computer literacy. The admin can efficiently monitor applications and make decisions using the system features and AI support. The farmer portal is designed for non-technical users with a step-by-step guided flow, inline validation messages, and clear status badges.

1.7 Requirements:

1.7.1 Functional Requirements

FR01: Farmer Registration & Input Validation

FR01-01	System shall collect full name, phone number (11 digits), CNIC (13 digits), address, city, field size, email, and password during registration
FR01-02	System shall validate phone number format (exactly 11 digits) using regex
FR01-03	System shall validate CNIC format (exactly 13 digits) using regex
FR01-04	System shall enforce password complexity (minimum 8 characters, 1 uppercase, 1 digit, 1 special character)
FR01-05	System shall detect and reject duplicate phone numbers and CNICs at the registration phase
FR01-06	System shall hash the password using bcrypt before storing it in the database

FR01-07	System shall display a real-time password strength indicator
---------	--

FR02: User Authentication & Session Management

FR02-01	System shall provide a login interface requiring the user's phone number and password
FR02-02	System shall authenticate farmers using their phone number and a bcrypt-verified password via secure PHP sessions
FR02-03	System shall display an error message if the authentication credentials do not match database records
FR02-04	System shall create a session upon successful login
FR02-05	System shall redirect unauthenticated users to the login page
FR02-06	System shall destroy the session and redirect to login page on logout

FR03: Farmer Profile Management

FR03-01	System shall allow farmers to view their personal details, farm information, and account statistics on a profile page
FR03-02	System shall allow farmers to upload or update their profile photo

FR04: Insurance Plan Exploration

FR04-01	System shall display all available insurance plans to all including public, unauthenticated visitors
FR04-02	System shall display explicit plan metrics for public viewing: premium rate, coverage level, applicable crops, and deductible
FR04-03	System shall allow farmers to navigate to the application form from a plan card

FR05: Insurance Plan Application Submission

FR05-01	System shall allow authenticated farmers to apply for a selected insurance plan via a structured multi-section form
FR05-02	System shall automatically pre-fill the farmer's personal details directly from the database into the insurance application form
FR05-03	System shall allow selection of policy duration (Kharif or Rabi)
FR05-04	System shall collect land details: district, bank account, insured area, irrigation type
FR05-05	System shall collect crop details: crop type, sowing date, expected yield, historical yield, seed variety, certified seed
FR05-06	System shall validate that insured area does not exceed farmer's total field size
FR05-07	System shall prevent duplicate applications for the same crop and season
FR05-08	System shall automatically calculate sum insured, coverage amount, final premium, and guaranteed production upon submission
FR05-09	System shall redirect to the invoice page after successful submission
FR05-10	System shall allow the farmer to download and print the premium invoice

FR06: Premium Payment Submission

FR06-01	System shall allow farmer to upload payment proof (JPG, PNG, WEBP, or PDF)
FR06-02	System shall validate uploaded file type and size (maximum 5 MB)
FR06-03	System shall capture transaction ID and payment date
FR06-04	System shall update application status to “Payment Submitted” after successful upload

FR07: Farmer Payment History

FR07-01	System shall allow farmer to view a history of all premium payment records including plan name, crop, premium amount, transaction ID, payment date, and payment status
FR07-02	System shall provide a link to view the invoice for each payment record

FR08: Damage Claim Filing

FR08-01	System shall allow farmers to file damage claims exclusively on policies that hold an active status
FR08-02	System shall collect loss date, damaged area, estimated loss, and loss reason
FR08-03	System shall strictly require exactly six crop evidence images, from different angles, to be uploaded per individual claim submission
FR08-04	System shall utilize client-side SHA-256 hashing to detect and instantly reject duplicate uploaded images before submission
FR08-05	System shall enforce a maximum limit of two claims per policy in case or rejected status and completely prohibit new claim submission on policies containing an already approved claim status

FR09: AI Based Crop Damage Analysis

FR09-01	System shall transmit each of the six uploaded evidence images to the external AI microservice for classification
FR09-02	System shall receive a “damaged” or “healthy” classification and confidence score for each image
FR09-03	System shall automatically compute the cumulative overall damage percentage and display the damage assessment result to the farmer. The claim is submitted for admin review and final decision regardless of damage count
FR09-04	System shall store all detailed AI analysis metrics (predicted class, confidence score, and calculated damage percentage) inside the database paired directly with the corresponding claim record
FR09-05	System shall display per-image AI verdicts and overall eligibility to the farmer before final submission
FR09-06	System shall generate a downloadable and print-friendly AI damage assessment report for each claim

FR10: Admin Authentication

FR10-01	System shall allow admin to login using username and password
FR10-02	System shall verify password against bcrypt hash stored in the users table
FR10-03	System shall protect all admin pages with session authentication check

FR11: Admin Dashboard

FR11-01	System shall feature a centralized administrative dashboard displaying live Key Performance Indicator (KPI) counters tracking: Total Farmers, Total Policies, Pending Claims, and Approved Claims
FR11-02	System shall display charts showing monthly applications and claims (last 6 months)
FR11-03	System shall display plan distribution and claim status distribution charts
FR11-04	System shall display a table of the five most recent applications

FR12: Admin Farmer Accounts Management

FR12-01	System shall allow admin to view all registered farmer accounts
FR12-02	System shall allow admin to add new farmer accounts manually
FR12-03	System shall allow admin to update farmer account details
FR12-04	System shall allow admin to delete farmer accounts

FR13: Admin Insurance Plan Management

FR13-01	System shall allow admin to create insurance plans with name, status, applicable crops, description, coverage level, premium rate, deductible rate, coverage type, and unit structure
FR13-02	System shall allow admin to update and delete existing plans
FR13-03	System shall allow admin to set market rates per crop for premium calculation

FR14: Admin Application & Payment Management

FR14-01	System shall display all farmers insurance applications with status badges
FR14-02	System shall allow admin to view uploaded payment proof
FR14-03	System shall allow admin to verify payment and activate the policy
FR14-04	System shall allow admin to reject payment with a reason shown to the farmer
FR14-05	System shall send a notification to the farmer upon payment verification or rejection
FR14-06	System shall allow admin to view and print the premium invoice for any application

FR15: Admin Claims Management

FR15-01	System shall display all claims with AI assessment results and report
FR15-02	System shall allow admin to view the detailed AI damage assessment report for any claim

FR15-03	System shall allow admin to approve or reject claims
FR15-04	System shall allow admin to mark a claim as “Under Review” for manual inspection
FR15-05	System shall allow admin to issue payout by uploading a payout receipt
FR15-06	System shall send a notification to the farmer for each claim status change
FR15-07	System shall allow the farmer to view and download the payout receipt once issued

FR16: Payout Processing

FR16-01	Admin shall manually transfer payout amount to farmer’s bank account
FR16-02	System shall allow admin to mark payout as “Payout Sent”
FR16-03	System shall update the farmer’s claim history with payout details

FR17: Farmer Notification System

FR17-01	System shall display an unread notification count badge on the navbar bell icon
FR17-02	System shall show the last 8 notifications in a dropdown panel
FR17-03	System shall allow farmers to mark individual or all notifications as read
FR17-04	System shall provide deep links in notifications directing to the relevant page

FR18: Password Recovery

FR18-01	System shall verify farmer identity using phone number and CNIC
FR18-02	System shall allow farmer to set a new password after identity verification
FR18-03	System shall enforce the same password complexity requirements as registration

FR19: Contact Form

FR19-01	System shall provide a contact form allowing visitors to submit their name, email, phone, and query
FR19-02	System shall store submitted contact inquiries in the database
FR19-03	System shall send an email notification to management via EmailJS upon successful contact form submission

1.7.2 Non- Functional Requirements

NFR01: System shall be accessible 24/7 through any modern web browser (Chrome, Firefox, Edge) when hosted on a live server.

NFR02: System shall have two types of users: farmer (user) and admin.

NFR03: All farmer and admin passwords shall be stored as bcrypt hashes, plaintext passwords shall never be stored or logged.

NFR04: All authenticated pages shall run PHP session guards to intercept unauthenticated users and immediately redirect them back to the login page.

NFR05: The AI analysis service shall return classification results within 30 seconds on a localhost deployment under normal CPU conditions.

NFR06: The user interface shall be entirely responsive, scaling perfectly across mobile, tablet, and desktop viewports utilizing a Bootstrap 5 layout grid.

NFR07: The automated AI classification engine shall natively support a minimum of two primary crop domains: Wheat and Rice.

NFR08: Uploaded crop evidence images shall be stored server-side using unique timestamped filenames to completely eliminate overwritten file collisions.

NFR09: All database queries shall use `mysqli_real_escape_string` to mitigate SQL injection.

NFR10: The system shall support both Kharif and Rabi agricultural seasons in plan applications.

NFR11: System shall respond to all page requests within 3 seconds under normal load conditions.

NFR12: System shall cast all ID parameters from GET/POST to integer type before database use.

NFR13: System shall validate file type and size for all file upload features before storing files on the server.

NFR14: System shall not allow a farmer to view another farmer's data (data isolation enforced server-side by session farmer_id comparison in all queries).

NFR15: The PHP cURL client invoking the AI analysis service shall be configured with a request timeout of 120 seconds to accommodate worst-case response times.

NFR16: System shall display informative flash messages (success/error) for all user actions.

1.7.3 Hardware Requirements

The following minimum hardware specifications are required to run the KissanSure web application on the user's (farmer's) device:

- **Processor:** Intel Core i3 (1.8 GHz) or equivalent / ARM Cortex-A53 (smartphone)
- **RAM:** 2 GB or more
- **Hard Disk:** 500 MB free space for browser cache
- **Camera:** Required for capturing crop images (minimum 2MP)
- **Internet Connection:** Stable internet connection (Broadband or 4G mobile data)
- **Display:** 1024×768 resolution (desktop) or 360×640 (smartphone minimum)

Server/development hardware requirements:

- **Processor:** Intel Core i5 or higher (for running AI model inference on CPU)
- **RAM:** 8 GB or more
- **Hard Disk:** 20 GB free space
- **Network:** Broadband or 4G mobile data

1.7.4 Software Requirements

End-user software requirements:

- **Operating System:** Windows 10/11, macOS, Android or iOS

- **Browser:** Google Chrome 100+, Mozilla Firefox 100+, or Microsoft Edge 100+ (any modern browser with JavaScript enabled)
- **PDF Viewer:** Built-in browser PDF viewer (for invoice and payout receipt downloads)

Server/development software requirements:

- **Operating System:** Windows 10
- **XAMPP (local):** Version 8.x
- **Web Server:** Apache 2.4 or higher (included in XAMPP)
- **PHP Version:** 8.0 or higher
- **Database:** MySQL 8.0 or higher
- **Python:** 3.8 or higher
- **Python Packages:** FastAPI, Uvicorn, PyTorch 2.0+, TorchVision, Pillow, python-multipart
- **IDE:** VS Code or any code editor

1.8 Stakeholders:

In the proposed AI-Based Crop Insurance System for Farmers, the following stakeholders are involved:

Primary Stakeholders:

- **Farmers (End Users):** Farmers are the most important stakeholders and the primary users of the system. They interact with the platform to register accounts, browse available insurance plans, submit insurance applications, upload payment proofs, file damage claims with photographic evidence, and receive AI-powered damage assessments. Their financial security depends directly on the system's accuracy and reliability. Their ease of use and satisfaction are critical to the success of the platform.
- **Insurance Administrators:** Administrators are responsible for the day-to-day management of the platform. They review and verify farmer applications, approve or reject payment submissions, manage the claims workflow, issue payouts, and configure insurance plans and market rates. They rely on the system's AI-generated damage assessment reports to support their decision-making on claim approvals.

Secondary Stakeholders:

- **Insurance Company / Management:** The insurance company or management body that offers crop insurance products through the platform is a key secondary stakeholder. They are interested in portfolio analytics, premium collection, risk management, and financial performance.
- **Agricultural Banks:** Agricultural banks are involved in the financial transactions that the system facilitates. Farmers transfer insurance premiums from their bank accounts and receive payouts into the same accounts.

- **Government / Regulatory Bodies:** The data collected by the platform regarding crop damage, insurance coverage, and payout distribution, can support policy planning related to agricultural risk management and farmer welfare programs at the provincial or national level.
- **Development Team:** The development team is responsible for designing, developing, testing, and maintaining the system. Their interest lies in ensuring the system meets all functional and non-functional requirements, operates securely, and is delivered within the project timeline.
- **Future Researchers and Students:** Future students and researchers can also benefit from the project as a learning resource for web development, AI integration, and insurance management systems.

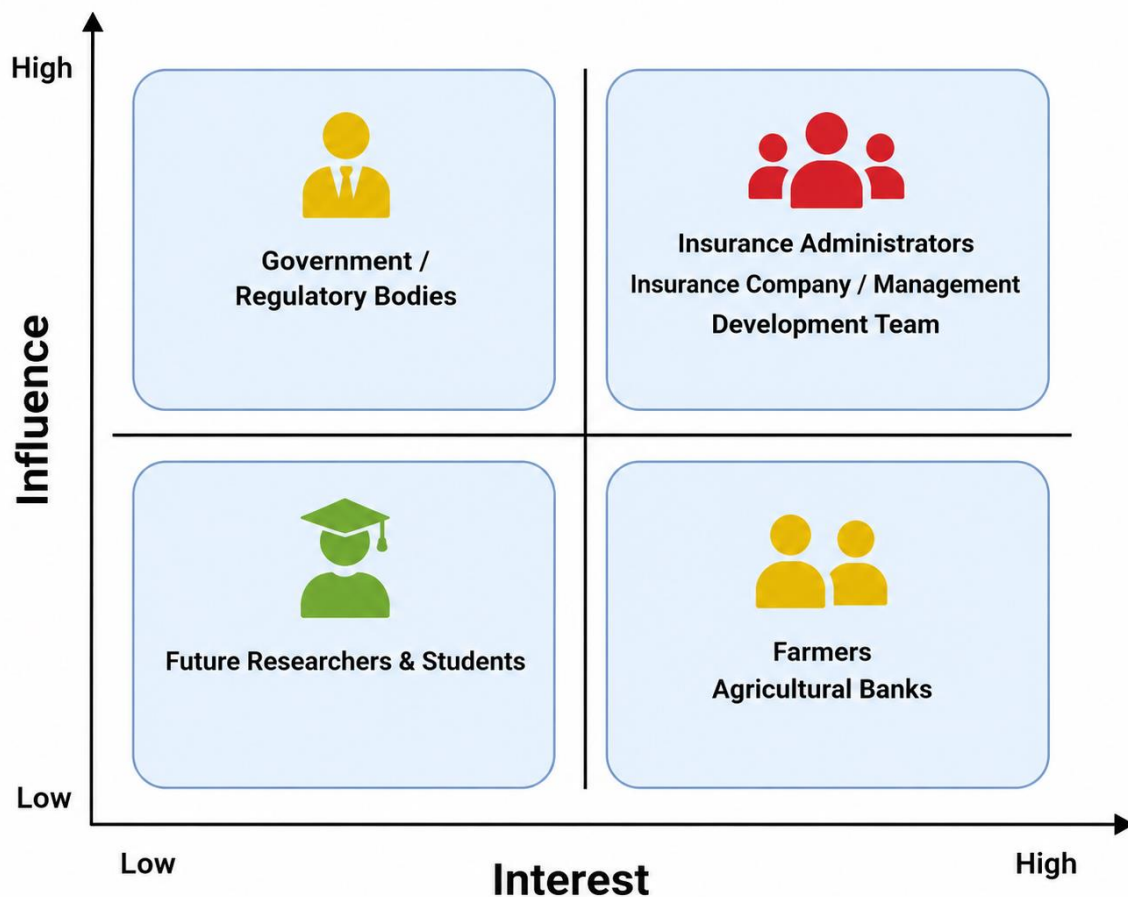


Figure 1.1 Stakeholders

Chapter 2 – MATERIALS & METHODS

2.1 Process Model:

The proposed system follows the **Rapid Application Development (RAD)** process model. RAD is an incremental and iterative software development approach that focuses on rapid prototyping, early customer feedback, and fast delivery of functional modules. Rather than waiting for the entire system to be completed before presenting it to the users, RAD develops components in smaller cycles, allowing continuous refinement and improvement. This model is especially useful when the project requires quick development, evolving requirements, and active involvement of stakeholders.

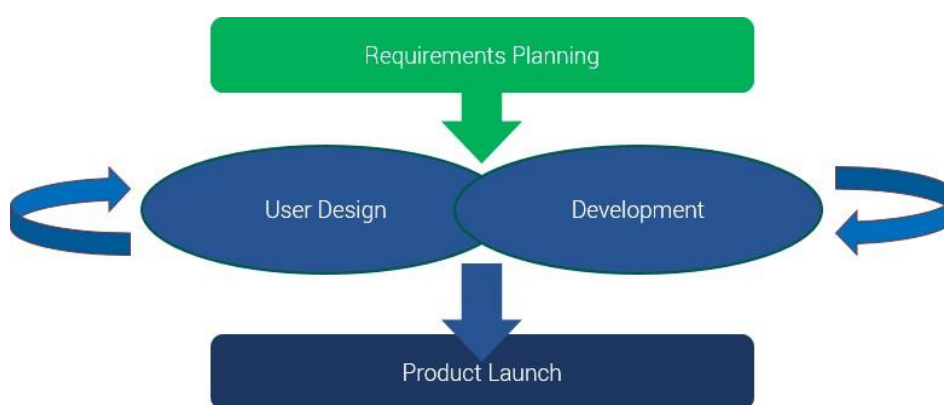


Figure 2.1: RAD Model

The project KissanSure was divided into three parallel development tracks: (1) the farmer-facing portal, (2) the administrative panel, and (3) the AI microservice. Each track was developed iteratively with regular integration testing. The admin portal was developed first to establish the core data model, followed by the farmer panel, and then the AI microservice was integrated in the final sprint. This approach allowed continuous feedback and rapid correction without a lengthy requirements-freeze phase.

RAD Phases Applied to the Project

Planning Requirements

In this phase, the core objectives and functional requirements of the system were identified. This included understanding the needs of farmers, insurance workflows, claim management processes, and AI-based damage assessment requirements. The scope, system modules, limitations, and user roles (farmer and admin) were finalized during this stage.

User Design

During this phase, multiple user interface prototypes were created for farmer dashboards, insurance purchasing forms, and admin panels. Feedback from the supervisor was gathered to

refine navigation flow, layout, and user experience. The design of the AI output report interface was also drafted in this stage.

Rapid Construction (Development)

In this stage, the actual development of system modules was carried out in iterative cycles. The development activities included:

- Web system was developed in PHP
- Backend AI integration was developed in Python (FastAPI)
- AI models (MobileNetV2 for classification) were trained and tested
- Database was implemented using MySQL

Each prototype was reviewed, refined, and improved based on continuous feedback.

Product Launch / Deployment

After all modules were developed and integrated, the complete system was tested for functionality, AI accuracy, usability, and performance. The final version was deployed in a working environment where both farmer and admin workflows operate smoothly. User acceptance testing and final refinements were conducted in this phase.

Table 2.1: Comparison of RAD Phases and Their Application in the Project

RAD Phase	Description	Application in FYP	Outcome
Requirements Planning	Identify scope & features	Defined modules, AI objectives, workflows	Clear project scope
User Design	Create UI prototypes, gather feedback	Dashboard, claim forms, admin UI prototypes	Improved and user-friendly interface
Rapid Construction	Iterative development of modules	PHP web system + Python AI + MySQL database	Functional prototypes delivered fast
Product Launch	Test, refine, integrate, deploy	Final system testing and deployment	Working model ready for presentation

2.2 Tools & Technologies

The following tools and technologies were used in the development of KissanSure:

- **PHP 8.2:** Server-side scripting language for all backend logic, database operations, and session management
- **MySQL:** Relational database management system storing all application data
- **MySQLi Extension:** PHP extension used for database queries (procedural API)
- **Apache (XAMPP):** Local web server for development and testing
- **HTML, CSS & JavaScript:** Used for designing, structuring and styling the front-end webpages.

- **Bootstrap 5.3.3:** CSS framework providing responsive grid layout, components, and utility classes
- **JavaScript (ES6):** Client-side scripting for form validation, AJAX calls, AI analysis, and dynamic UI updates
- **Web Crypto API (SHA-256):** Used client-side for detecting duplicate uploaded images
- **Python 3.8+:** Programming language for the AI microservice
- **FastAPI:** Modern Python web framework for building the AI REST API
- **Uvicorn:** ASGI server for running the FastAPI application
- **PyTorch 2.0+ / TorchVision:** Deep learning framework and model library for AI inference
- **MobileNetV2:** Lightweight pre-trained CNN used as the binary crop damage classifier
- **Pillow (PIL):** Python image processing library for preprocessing uploaded images
- **VS Code:** Integrated Development Environment (IDE) used for all development
- **Git:** Version control system and remote repository hosting for source code management
- **Google Colab:** Cloud-based Jupyter notebook environment (Tesla T4 GPU) used for training and evaluating AI models

2.3 Design:

The design section documents the system architecture, database design, and UML diagrams that describe the structure and behaviour of KissanSure.

2.3.1 Use Case Diagrams:

A use case diagram depicts the relationships between actors and use cases within the KissanSure system boundary. In KissanSure, each use case corresponds directly to a core functional requirement.

Actors:

The KissanSure system has three actors:

- **Farmer:** The primary end-user of the system. A registered farmer interacts with the platform to purchase crop insurance and file damage claims.
- **Admin:** The insurance administrator who manages the platform, verifies farmer payments, reviews AI-analyzed claims, and issues payouts.
- **AI System (FastAPI Server):** An external computing entity (the Python AI microservice running on FastAPI) that receives crop images and returns damage classification results. It is an external system actor, not a human.

Use Cases:

The nine identified use cases of KissanSure are:

1. **Farmer Registration:** The farmer creates a new account by providing personal, contact, and farm details.
2. **Farmer Login:** The farmer authenticates into the system using a registered phone number and password.
3. **Buy Insurance Plan:** The farmer browses available plans, selects one, and submits an insurance application with crop and land details. The system auto-calculates the premium.
4. **Submit Payment Proof:** The farmer uploads proof of premium payment along with a transaction ID for admin verification.
5. **Admin Verify Payment & Activate Plan:** The admin reviews submitted payment proof and either activates the farmer's policy or rejects it with a reason.
6. **Submit Damage Claim:** A farmer with an active policy files a crop damage claim, uploading six photographic evidence images along with loss details.
7. **AI Damage Analysis:** The system automatically sends the six claim images to the AI microservice (FastAPI). MobileNetV2 classifies each image as damaged or healthy and returns a confidence score. The system aggregates results to determine claim eligibility and calculate the payout.
8. **Admin Review & Decide Claim:** The admin reviews the farmer's claim and the AI-generated damage report, then approves, rejects, or places the claim under review.
9. **Process Claim Payout:** After approving a claim, the admin issues the compensation payout and uploads a receipt. The farmer is notified and can download the payout receipt.

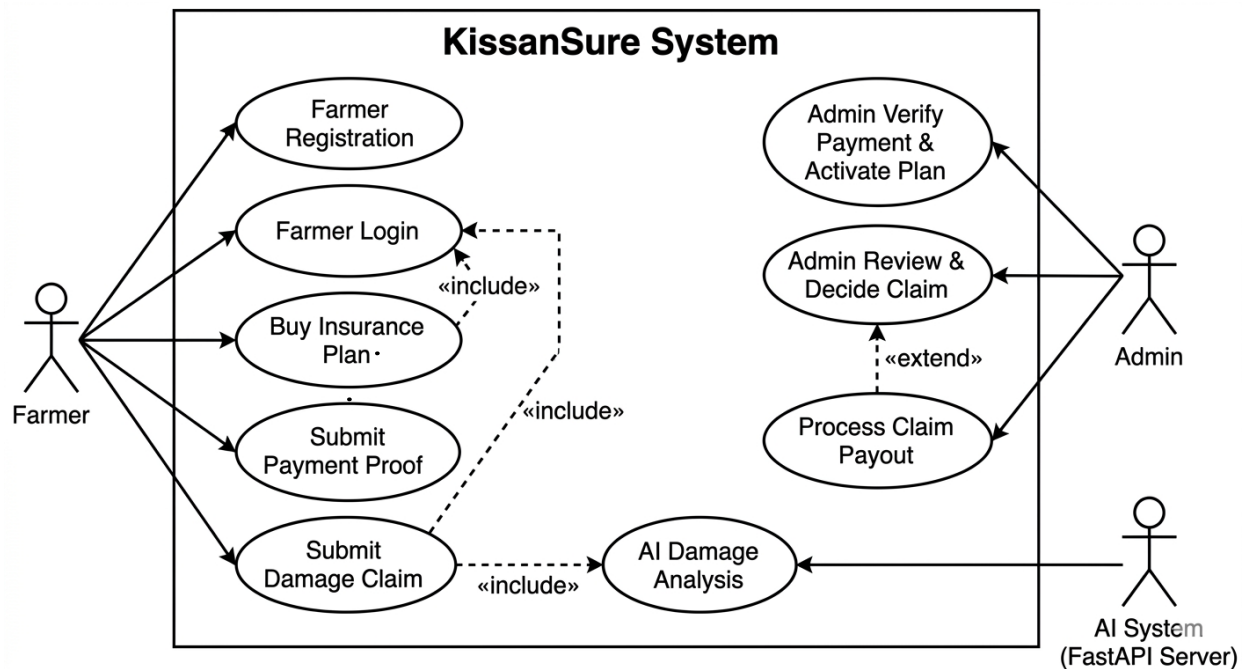


Figure 2.2 Use Case Diagram

2.3.2 Usage Scenario

A usage scenario is a text-based representation of a use case, describing the step-by-step interactions between the actor and the system. The following tables present usage scenarios for all nine use cases of KissanSure identified in Section 2.3.1.

Table 2.2: Farmer Registration

Use Case Title	Farmer Registration		
Abbreviated Title	F-REG		
Use Case Id	UC-01		
Requirement Id	FR01		
Description: A new user registers a farmer account by providing personal, contact, and farm details. The system validates all inputs and creates the account securely.			
Pre Conditions:			
1. User is not already registered in the system. 2. Database is available.			
Task Sequence		Exceptions	
1. User navigates to the registration page			
2. User provides full name, father's name, phone number (11 digits), CNIC (13 digits), email, address, city, and field size			
3. User sets a password meeting complexity requirement (minimum 8 characters, 1 uppercase, 1 digit, 1 special character) and confirms it		Password does not meet complexity: real-time strength indicator shows unmet requirements	
4. System validates all field formats using regex		Phone or CNIC format invalid: System displays format error	
5. System checks for duplicate phone number and CNIC		Duplicate found: System displays "Phone number/CNIC already registered"	
6. System hashes password using bcrypt and creates farmer account			
7. System creates the farmer account and redirects to the login page with a success message			
Post Conditions:			
- Farmer account is created in the database. - Farmer is redirected to the login page with a success message.			
Authority: Unregistered User			
Modification history: 1.0			
Author: Ushba Zahid (2022-ag-7710), Izzah Khursheed (2022-ag-7726)			

Table 2.3: Farmer Login

Use Case Title	Farmer Login
Abbreviated Title	F-LOGIN
Use Case Id	UC-02
Requirement Id	FR02
Description: A registered farmer authenticates into the system using their phone number and password to access the farmer portal.	
Pre Conditions:	
1. Farmer is already registered in the system.	

2. Database is available.	
Task Sequence	Exceptions
1. Farmer navigates to the login page	
2. Farmer enters registered phone number and password	Empty credentials: System displays validation error
3. System queries database by phone number and verifies password against bcrypt hash	Phone not found or wrong password: System displays "Invalid phone number or password"
4. System creates session and redirects farmer to the home page	
Post Conditions: - Farmer session is established. - Farmer is redirected to the home page with their name displayed in the navigation bar.	
Authority: Registered Farmer	
Modification history: 1.0	
Author: Ushba Zahid (2022-ag-7710), Izzah Khursheed (2022-ag-7726)	

Table 2.4: Buy Insurance Plan

Use Case Title	Buy Insurance Plan
Abbreviated Title	BUY-PLAN
Use Case Id	UC-03
Requirement Id	FR04, FR05
Description: An authenticated farmer browses available insurance plans, selects a suitable plan, and submits an insurance application with crop and land details. The system auto-calculates the premium and generates an invoice.	
Pre Conditions: - Farmer is logged in. - At least one insurance plan is available. - Farmer does not have an existing active application for the same crop and season.	
Task Sequence	Exceptions
1. Farmer navigates to the Insurance Plans page	
2. Farmer views available plans showing premium rates, coverage levels, applicable crops, and deductible rates	
3. Farmer selects a plan and proceeds to the application form	
4. Farmer selects policy duration (Kharif or Rabi)	
5. Farmer fills in land details: district, bank account number, insured area, and irrigation type	Insured area exceeds field size: System displays validation error
6. Farmer fills in crop details: crop type, sowing date, expected yield, historical yield, seed variety	Duplicate application for same crop and season: System displays "You already have an active application for this crop and season"
7. System auto-calculates sum insured, coverage amount, final premium, and guaranteed production	
8. System saves application with "Pending Payment" status and redirects to invoice page	
Post Conditions: - Insurance application is created with "Pending Payment" status. - Farmer is redirected to the premium invoice page.	

Authority: Authenticated Farmer
Modification history: 1.0
Author: Ushba Zahid (2022-ag-7710), Izzah Khursheed (2022-ag-7726)

Table 2.5: Submit Payment Proof

Use Case Title	Submit Payment Proof
Abbreviated Title	PAY-PROOF
Use Case Id	UC-04
Requirement Id	FR06
Description: A farmer with a pending payment application uploads proof of premium payment along with a transaction ID for admin verification.	
Pre Conditions:	
1. Farmer is logged in.	
2. Farmer has at least one application with "Pending Payment" status.	
Task Sequence	Exceptions
1. Farmer navigates to the invoice page of a pending application	
2. Farmer uploads payment proof file (JPG, PNG, WEBP, or PDF, maximum 5 MB)	Invalid file type or size exceeds 5 MB: System displays validation error
3. Farmer enters transaction ID and payment date	No file selected: System prevents submission
4. System validates file type and size	
5. System stores payment proof and updates application status to "Payment Submitted"	
6. Admin checks the applications dashboard to view new payment submissions	
Post Conditions:	
– Payment proof is stored.	
– Application status changes to "Payment Submitted".	
– Admin can view the submission from the dashboard.	
Authority: Authenticated Farmer	
Modification history: 1.0	
Author: Ushba Zahid (2022-ag-7710), Izzah Khursheed (2022-ag-7726)	

Table 2.6: Admin Verify Payment & Activate Plan

Use Case Title	Admin Verify Payment & Activate Plan
Abbreviated Title	VERIFY-PAY
Use Case Id	UC-05
Requirement Id	FR14
Description: The admin reviews the farmer's submitted payment proof and either activates the policy or rejects the payment with a reason.	
Pre Conditions:	
1. Admin is logged in.	
2. At least one application with "Payment Submitted" status exists.	
Task Sequence	Exceptions
1. Admin navigates to the View Applications page	
2. Admin selects an application with "Payment Submitted" status	

3. System displays application details, coverage summary, and uploaded payment proof	No payment proof uploaded: System shows "No payment proof uploaded yet"
4. Admin reviews the payment proof	
5a. If payment is valid: Admin verifies payment. System activates policy, records activation date, and sends notification to farmer	
5b. If payment is invalid: Admin enters rejection reason and rejects payment. System reverts status to "Pending Payment", stores rejection reason, and sends notification to farmer	Rejection reason is empty: System prevents submission
Post Conditions: – Policy is either set to "Active" (verified) or returned to "Pending Payment" (rejected). – Farmer receives a notification.	
Authority: Admin	
Modification history: 1.0	
Author: Ushba Zahid (2022-ag-7710), Izzah Khursheed (2022-ag-7726)	

Table 2.7: Submit Damage Claim

Use Case Title	Submit Damage Claim
Abbreviated Title	SUB-CLAIM
Use Case Id	UC-06
Requirement Id	FR08
Description: A farmer with an active policy submits a crop damage claim by providing loss details and uploading exactly six photographic evidence images. The system runs AI analysis on the images before final submission	
Pre Conditions: 1. Farmer is logged in. 2. Farmer has at least one active insurance policy. 3. Farmer has not exceeded the claim limit (maximum two claims per policy; no approved claim exists on the same policy).	
Task Sequence	Exceptions
1. Farmer navigates to the damage claim filing page	
2. Farmer selects an active policy from the dropdown	No active policy: System displays message to first apply for insurance
3. System auto-fills the crop type of the selected policy	
4. Farmer enters loss date, damaged area, estimated loss, and loss reason	Damaged area exceeds insured area: System displays validation error
5. Farmer uploads exactly six photographic crop evidence images from different angles	Fewer than six images: Submit button remains disabled
6. System performs client-side SHA-256 hash check to detect duplicate images	Duplicate image detected: System displays error and rejects the duplicate
7. System triggers AI Damage Analysis (include: AI Damage Analysis)	AI service unavailable: System displays "AI service unavailable. Try again later."
8. System displays per-image AI verdicts and overall eligibility result to farmer	

9. Farmer confirms and submits the claim	Claim limit reached: System displays message that no further claim can be filed
10. System saves the claim record with AI results, damage percentage, and calculated payout	
Post Conditions: – Claim record is saved with AI analysis results, damage percentage, and calculated payout. – Farmer is redirected to claim history page.	
Authority: Authenticated Farmer with Active Policy	
Modification history: 1.0	
Author: Ushba Zahid (2022-ag-7710), Izzah Khursheed (2022-ag-7726)	

Table 2.8: AI Damage Analysis

Use Case Title	AI Damage Analysis
Abbreviated Title	AI-ANLYS
Use Case Id	UC-07
Requirement Id	FR09
Description: The system automatically transmits the six uploaded claim images to the FastAPI AI microservice. MobileNetV2 classifies each image as damaged or healthy with a confidence score. The system aggregates results to determine eligibility and calculate the payout.	
Pre Conditions: 1. Exactly six evidence images have been uploaded. 2. Crop type is known (wheat or rice). 3. FastAPI AI microservice is running and accessible.	
Task Sequence	Exceptions
1. System receives six uploaded images and crop type from the claim submission	
2. System sends each image sequentially to the FastAPI /analyze endpoint via cURL POST	FastAPI server unreachable: System returns "AI service unavailable. Try again later."
3. FastAPI loads the appropriate MobileNetV2 model (wheat or rice)	
4. Model preprocesses each image (resize to 224x224, normalize using ImageNet parameters)	
5. Model classifies each image as "damaged" or "healthy" and returns a confidence score	
6. System counts the number of images classified as damaged	
7. System calculates damage percentage: $(\text{damaged_count} / 6) \times 100$	
8. System calculates final payout: $\text{coverage_amount} \times \text{damage_percentage} \times (1 - \text{deductible_rate}\%)$	
9. System returns all per-image verdicts and overall damage percentage to the farmer interface. Submit button is enabled for farmer to confirm and submit.	
Post Conditions: – All six images are classified. – Damage percentage and calculated payout are ready for display and storage with the claim record. Farmer submits the claim for admin review.	
Authority: System (triggered automatically during claim submission)	

Modification history: 1.0

Author: Ushba Zahid (2022-ag-7710), Izzah Khursheed (2022-ag-7726)

Table 2.9: Admin Review & Decide Claim

Use Case Title	Admin Review & Decide Claim		
Abbreviated Title	REVIEW-CLM		
Use Case Id	UC-08		
Requirement Id	FR15		
Description: The admin reviews a submitted damage claim alongside the AI-generated damage report and decides to approve, reject, or place the claim under review.			
Pre Conditions:			
1. Admin is logged in.			
2. At least one claim with "AI Analyzed" status exists.			
Task Sequence			Exceptions
1. Admin navigates to the View Claims page			
2. Admin selects a claim for review			
3. System displays claim details, uploaded evidence images, and AI damage assessment report (per-image verdicts, damage percentage, calculated payout)			
4. Admin reviews all claim information and the AI report			
5a. Admin approves: System updates claim status to "Approved" and sends notification to farmer			
5b. Admin rejects: Admin enters rejection reason. System updates status to "Rejected" and sends notification to farmer			
5c. Admin marks under review: System updates claim status to "Under Review" for manual inspection			
Post Conditions:			
– Claim status is updated to Approved, Rejected, or Under Review.			
– Farmer receives a notification of the decision			
Authority: Admin			
Modification history: 1.0			
Author: Ushba Zahid (2022-ag-7710), Izzah Khursheed (2022-ag-7726)			

Table 2.10: Process Claim Payout

Use Case Title	Process Claim Payout	
Abbreviated Title	PROC-PAY	
Use Case Id	UC-09	
Requirement Id	FR16	
Description: After approving a claim, the admin transfers the payout amount to the farmer's bank account, uploads a payout receipt, and marks the payout as sent. The farmer is notified and can download the receipt.		
Pre Conditions: 1. Admin is logged in. 2. Claim status is "Approved".		
Task Sequence		Exceptions
1. Admin navigates to the approved claim		

2. Admin transfers the payout amount to the farmer's registered bank account (external bank transfer)	
3. Admin uploads the payout receipt to the system	No receipt uploaded: System prevents marking payout as sent
4. Admin marks the payout as sent	
5. System updates claim status to "Payout Sent", records payout date, and sends notification to farmer	
6. Farmer can view and download the payout receipt from their claim history	
Post Conditions: – Claim payout status is set to "Payout Sent". – Payout receipt is stored. – Farmer is notified and can download the receipt.	
Authority: Admin	
Modification history: 1.0	
Author: Ushba Zahid (2022-ag-7710), Izzah Khursheed (2022-ag-7726)	

2.3.3 Sequence Diagram:

In KissanSure, the main participants across all diagrams are the Farmer or Admin actor, the Browser (client side), the PHP Application Server (Apache), the MySQL Database, and the FastAPI AI Microservice where applicable. A separate sequence diagram is provided for each of the nine use cases identified in Section 2.3.1.

Farmer Registration

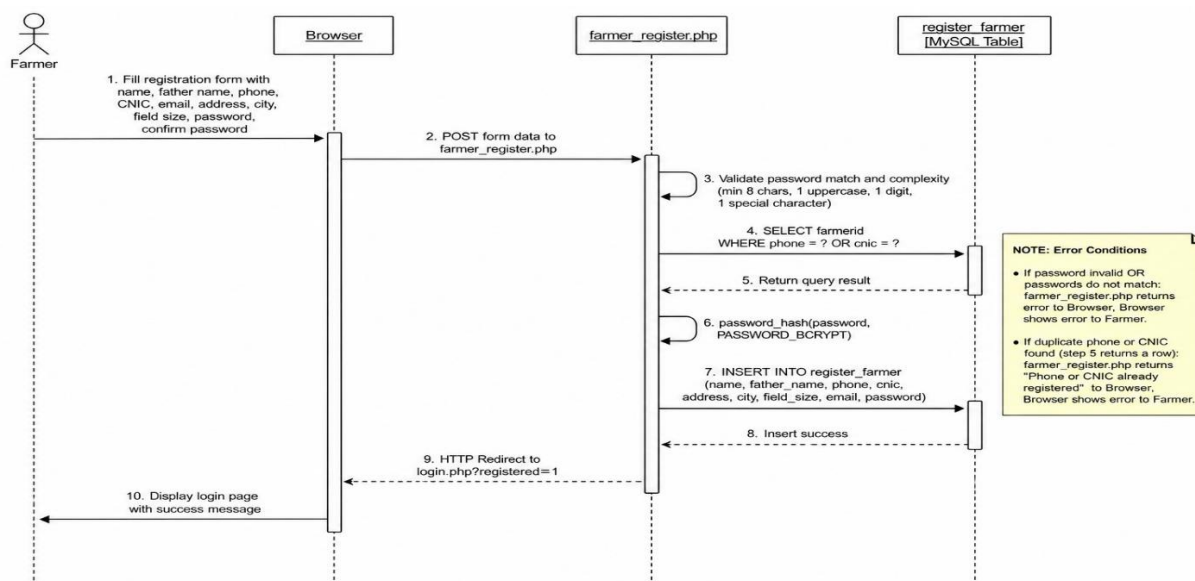


Figure 2.3 Farmer Registration Sequence Diagram

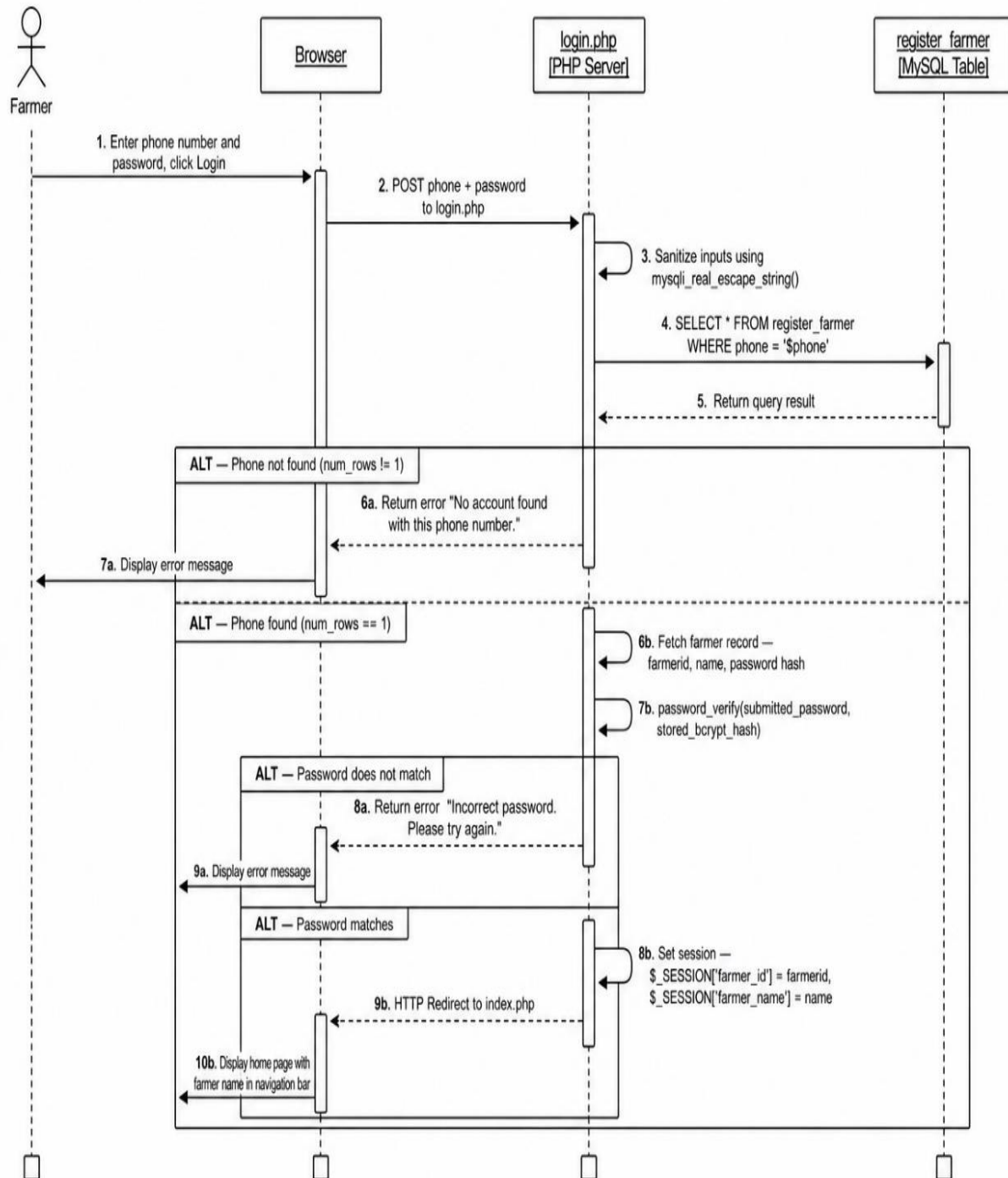
Farmer Login

Figure 2.4 Farmer Login Sequence Diagram

Buy Insurance Plan

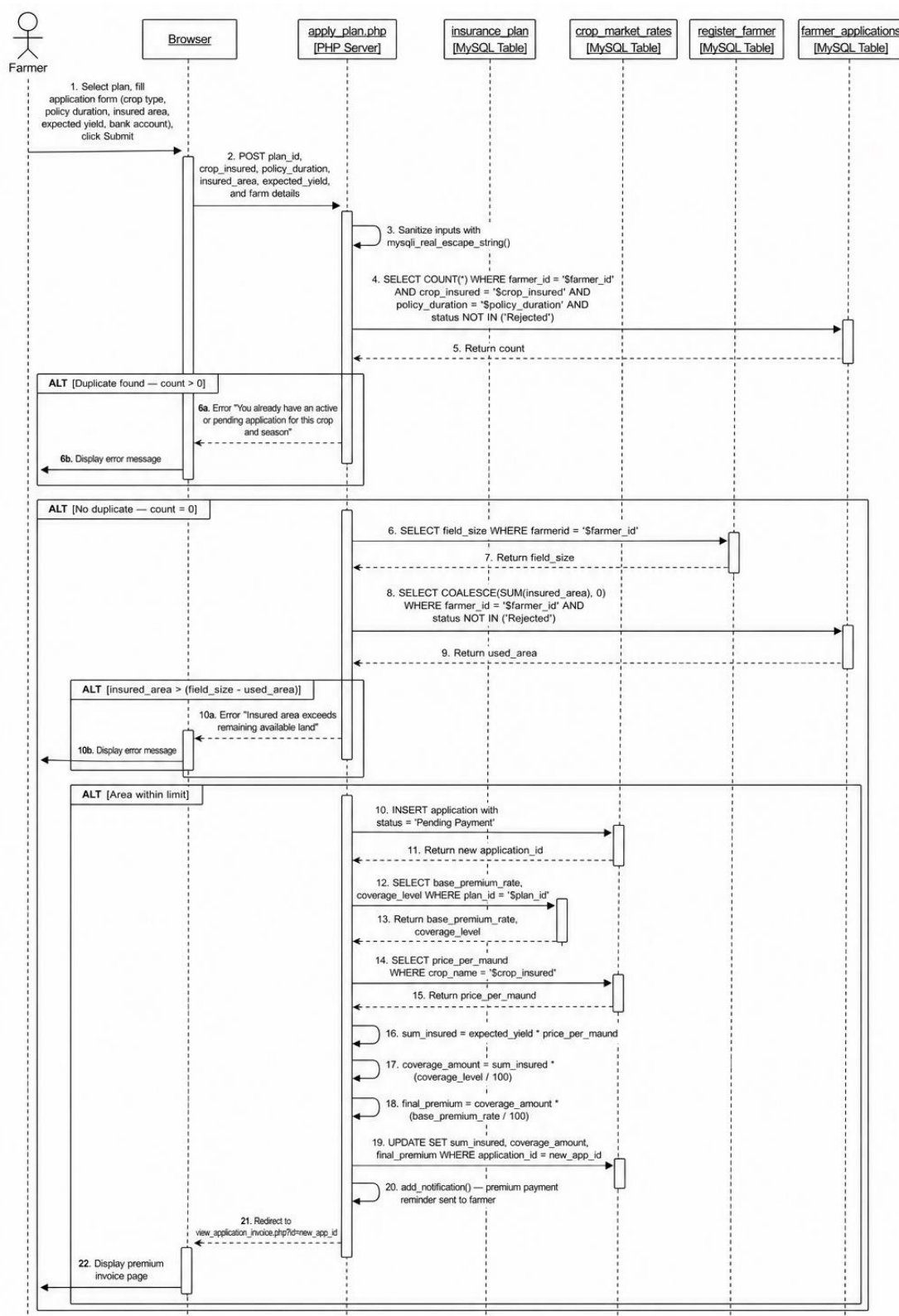


Figure 2.5 Buy Insurance Plan Sequence Diagram

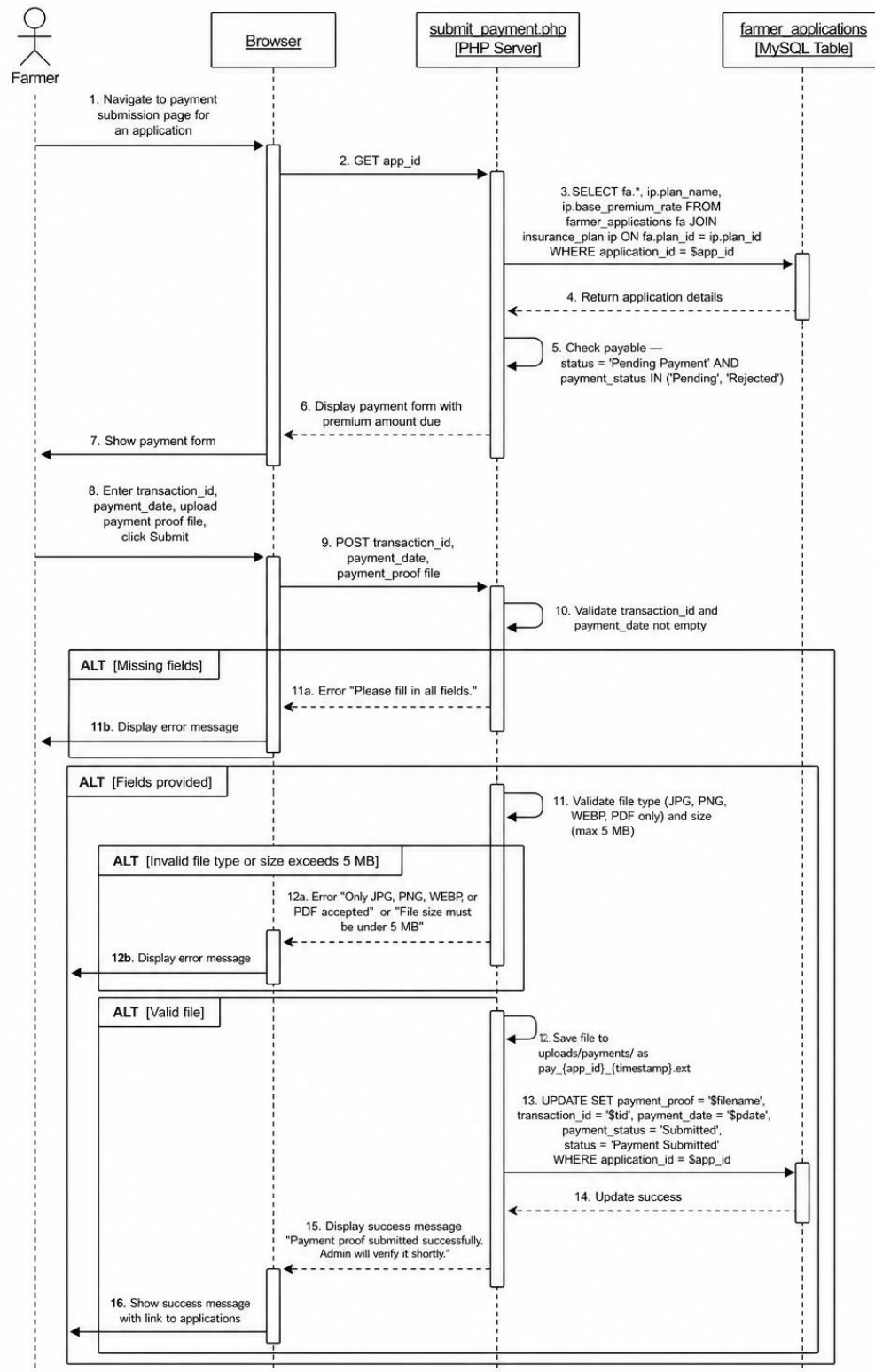
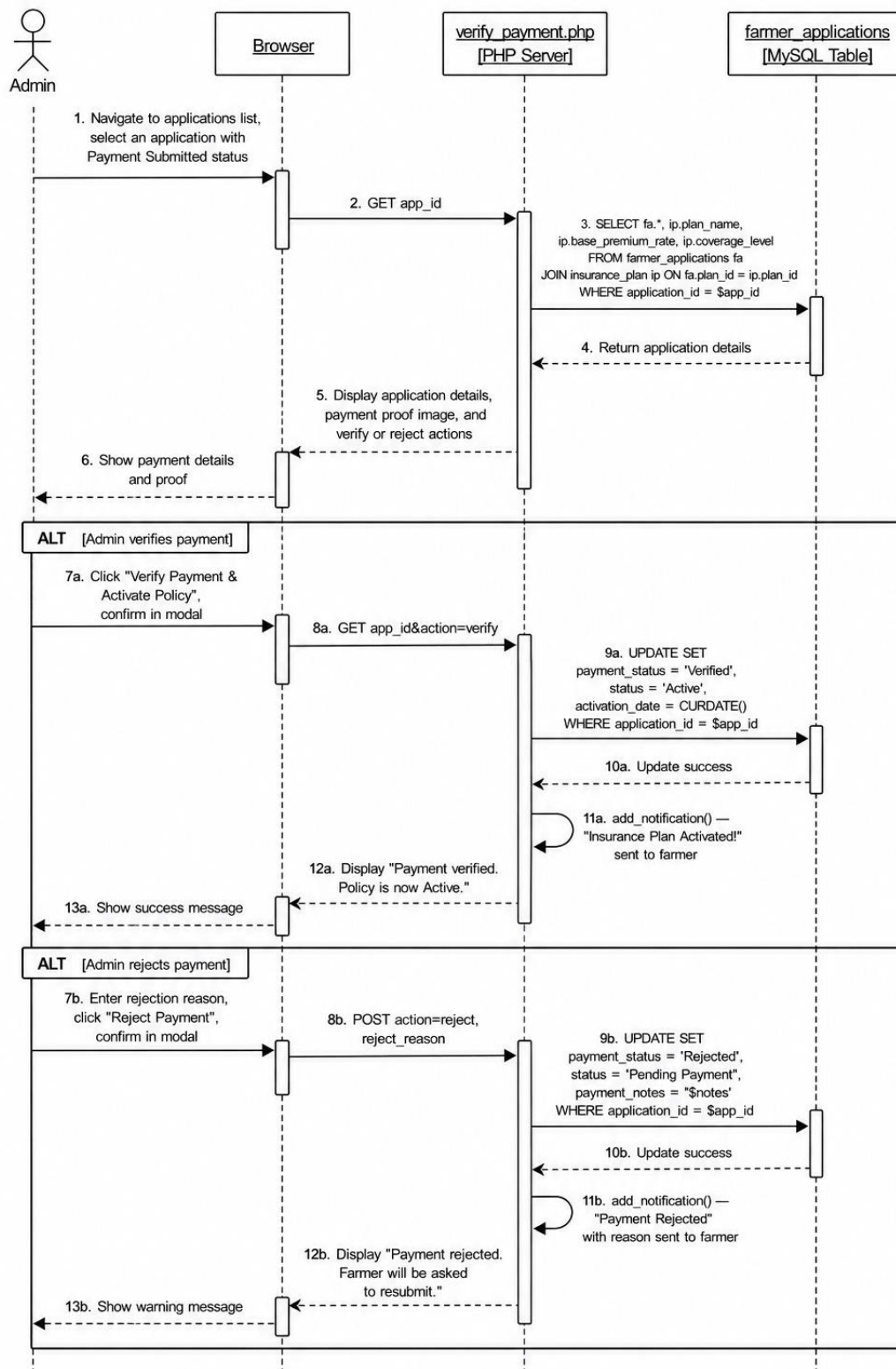
Submit Payment Proof

Figure 2.6 Submit Payment Proof Sequence Diagram

Admin Verify Payment & Activate Plan*Figure 2.7 Admin Verify Payment & Activate Plan Sequence Diagram*

Submit Damage Claim

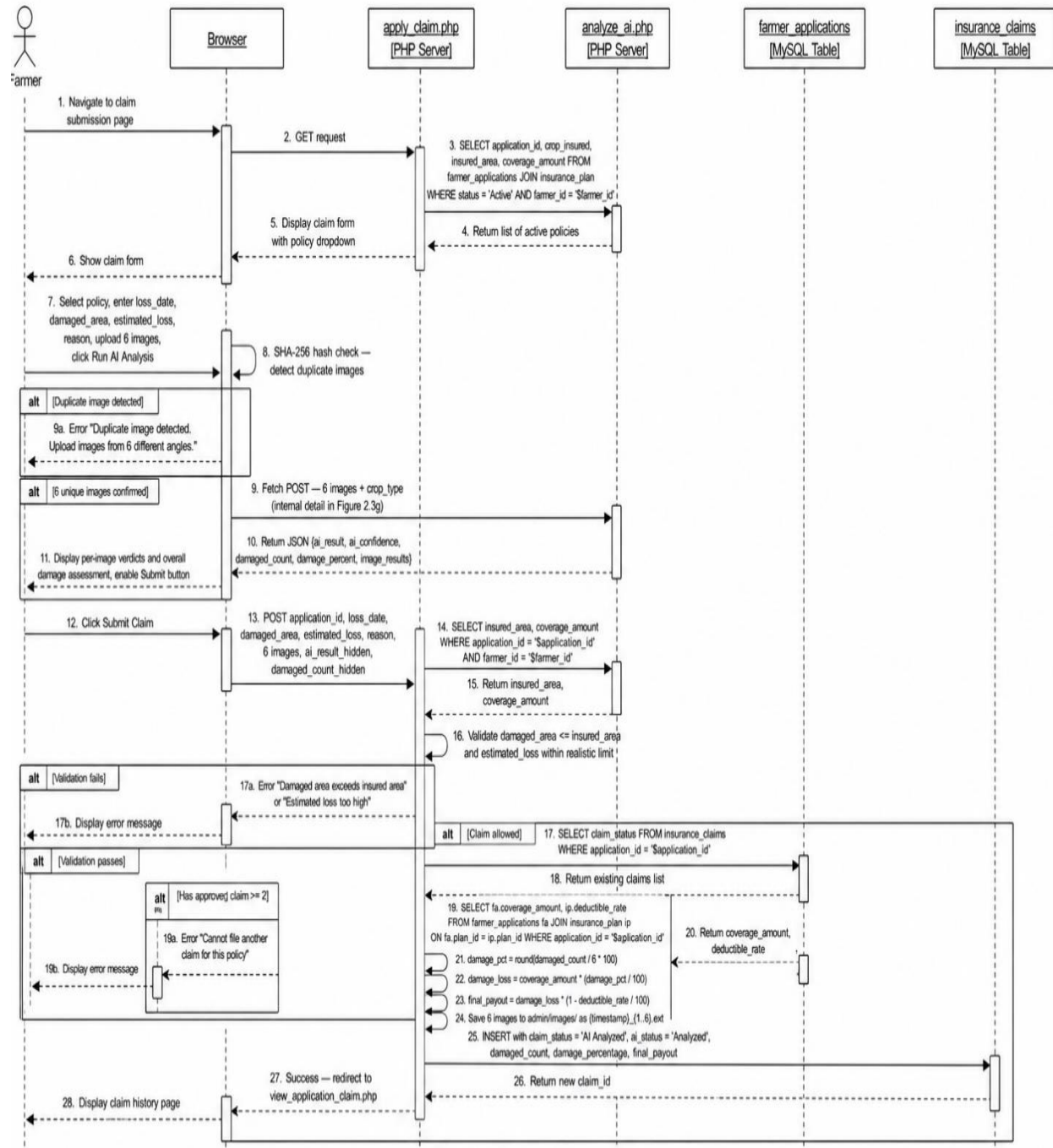


Figure 2.8 Submit Damage Claim Sequence Diagram

AI Damage Analysis

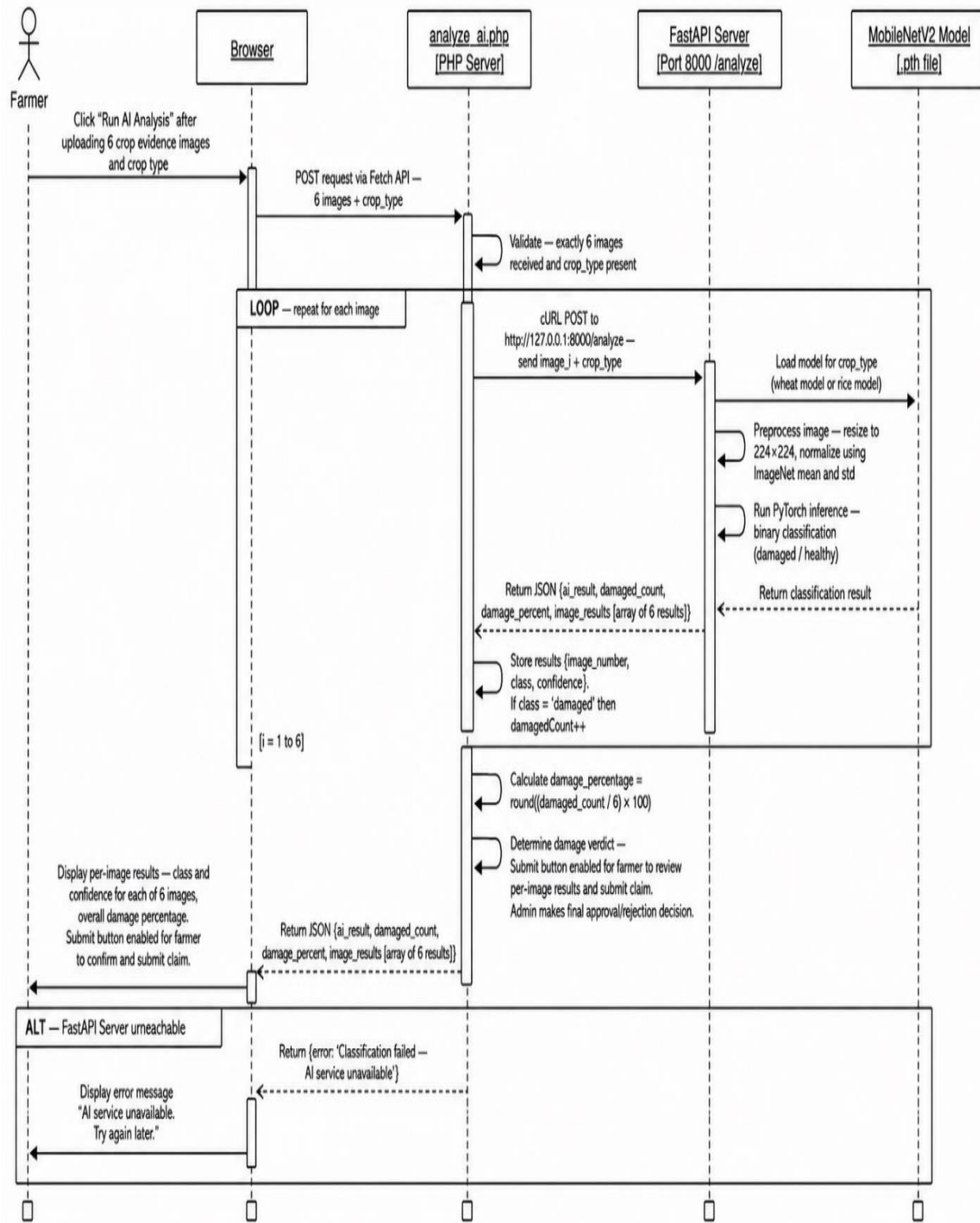


Figure 2.9 AI Damage Analysis Sequence Diagram

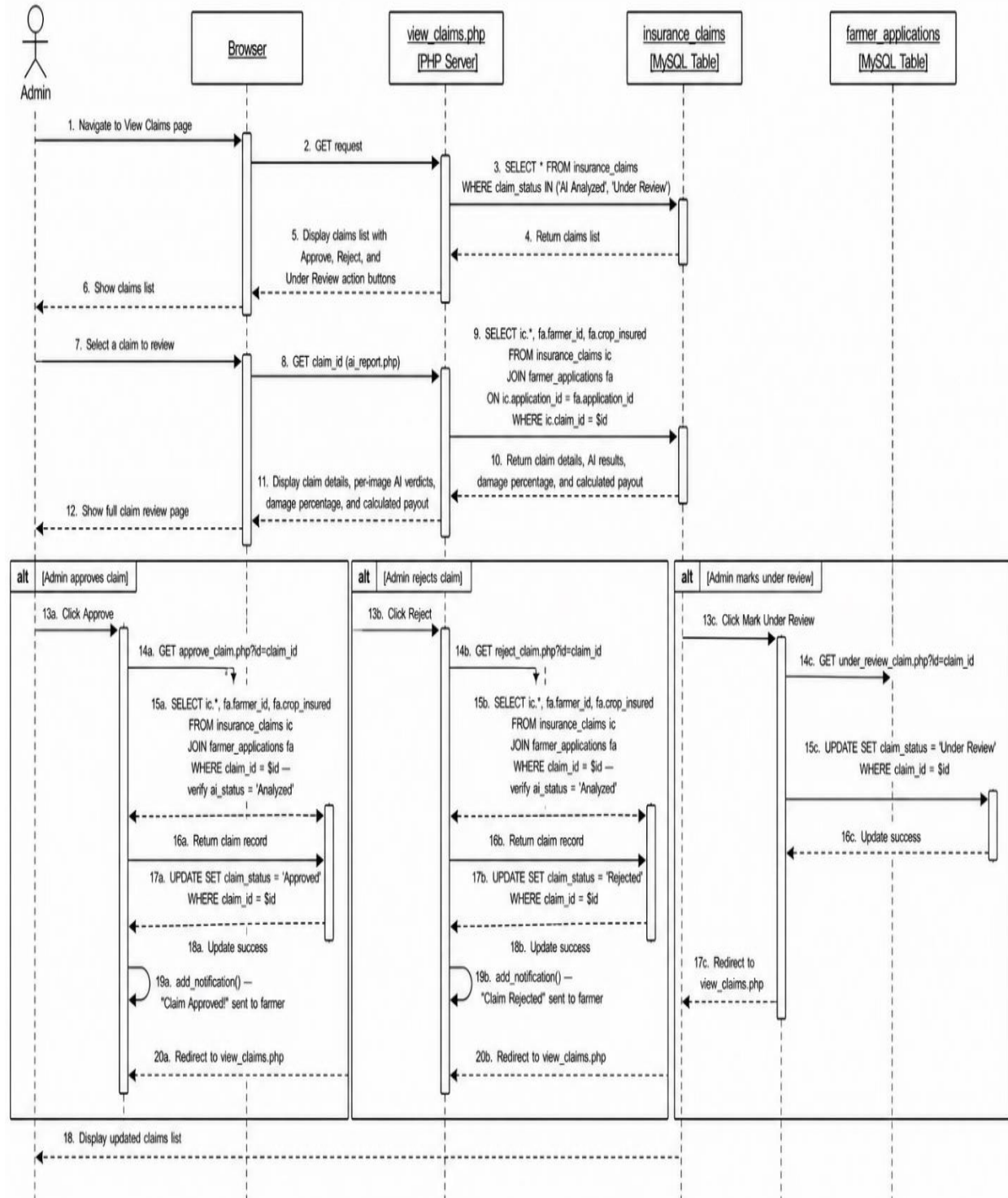
Admin Review & Decide Claim

Figure 2.10 Admin Review & Decide Claim Sequence Diagram

Process Claim Payout

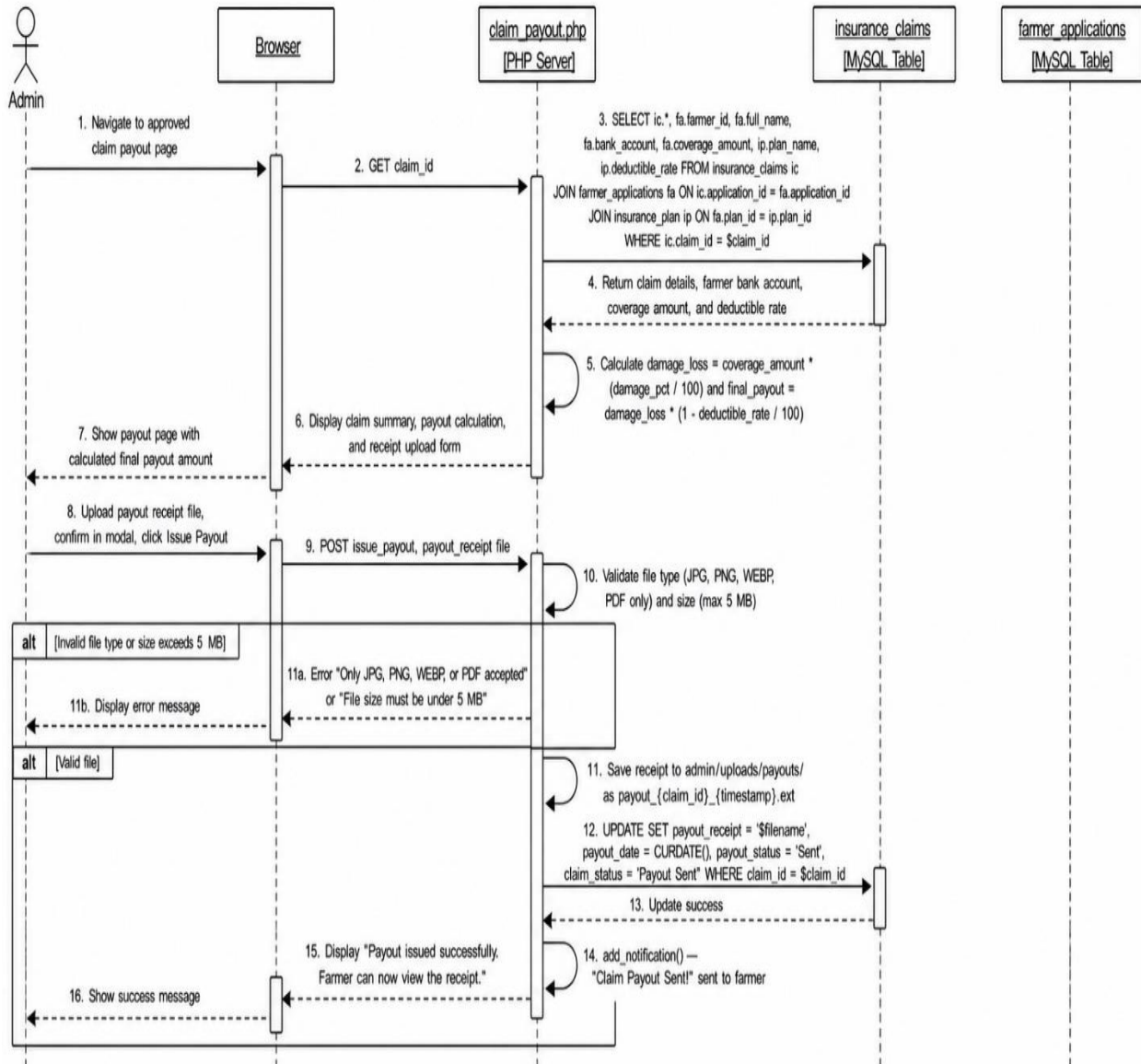


Figure 2.11 Process Claim Payout Sequence Diagram

2.3.4 Class Diagram:

A class diagram is a static structure diagram that describes a system by showing its classes, their attributes, operations, and the relationships among objects [5]. The classes are Farmer, Admin, InsurancePlan, FarmerApplication, InsuranceClaim, CropMarketRate, Notification, AIAnalyzer.

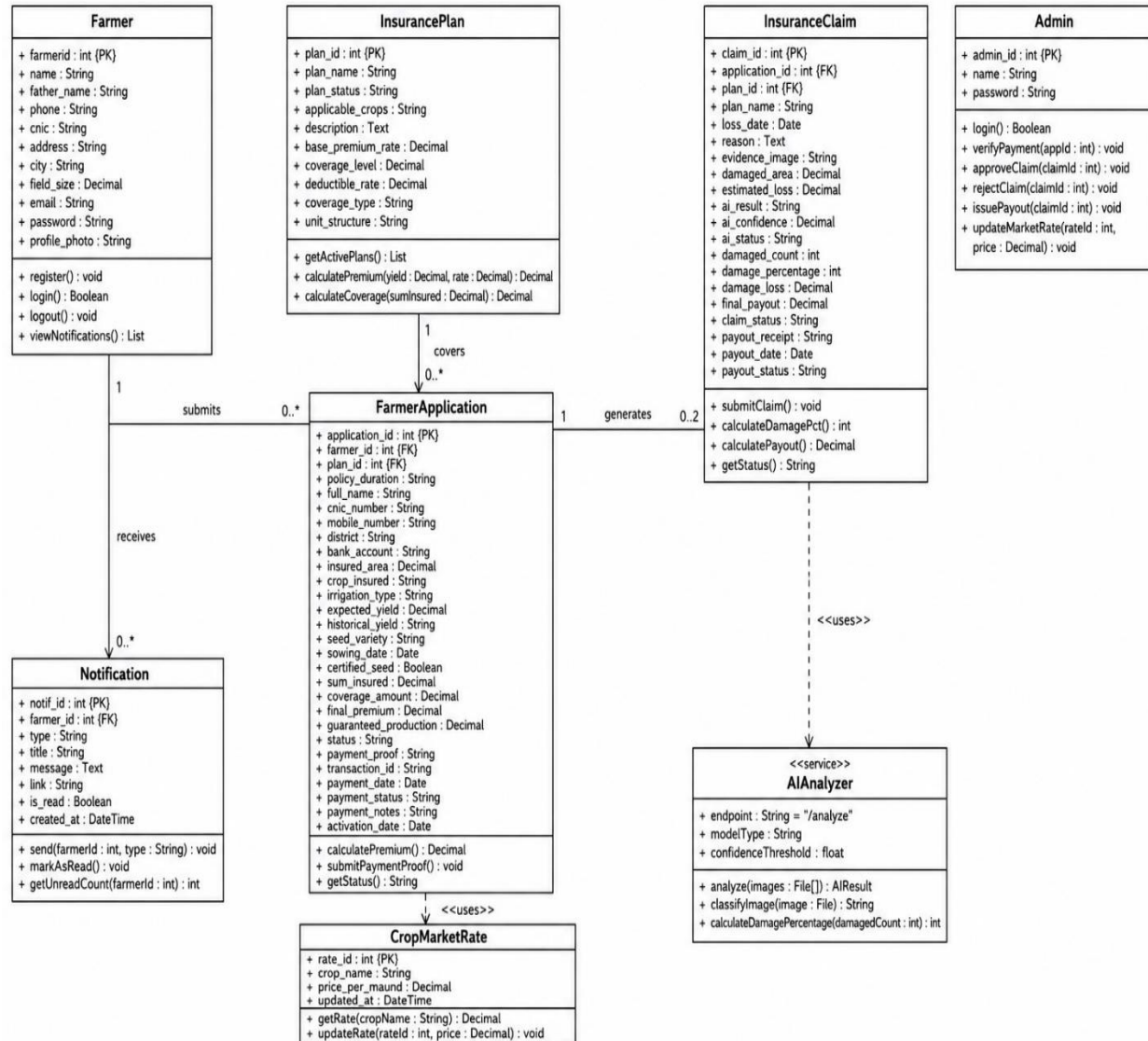


Figure 2.12 Class Diagram

2.3.5 Data Flow Diagram:

A data flow diagram (DFD) maps the flow of information through a system, showing how data moves between external actors, internal processes, and data stores [6]. It is structured in levels, with each level providing a deeper view of the system's functionality.

KissanSure uses a four-level DFD structure. The Context Diagram presents the entire system as a single process. The Level 0 DFD decomposes it into five core functional processes [7]. The Level 1 DFD further details the Claim and AI Processing workflow, which is the most complex process

in the system. The Level 2 DFD provides the deepest decomposition, breaking down the AI analysis and payout calculation sub-process into its individual steps.

Context Diagram:

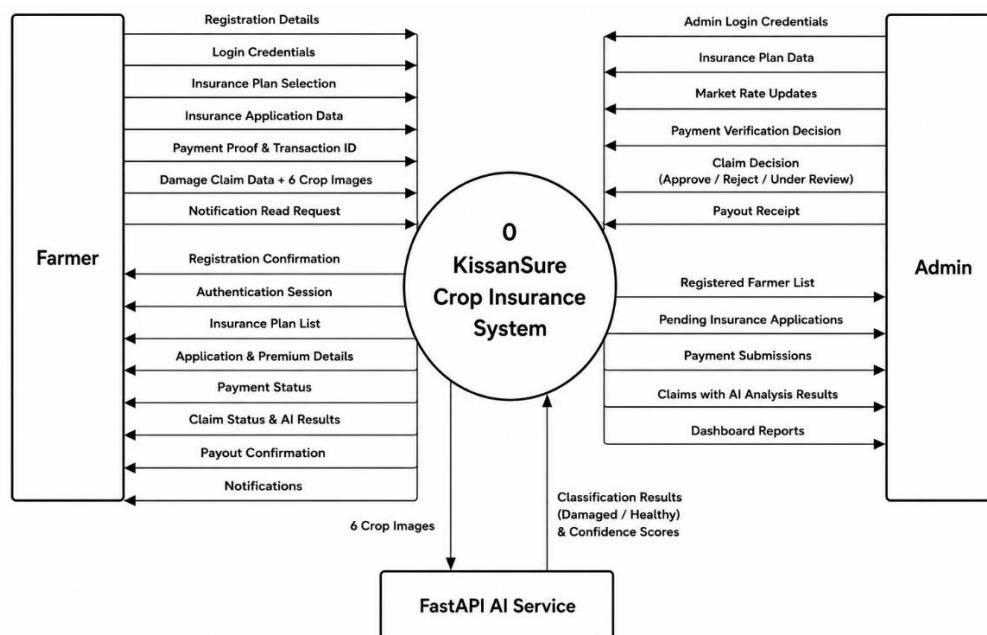


Figure 2.13 Context Diagram

Level 0:

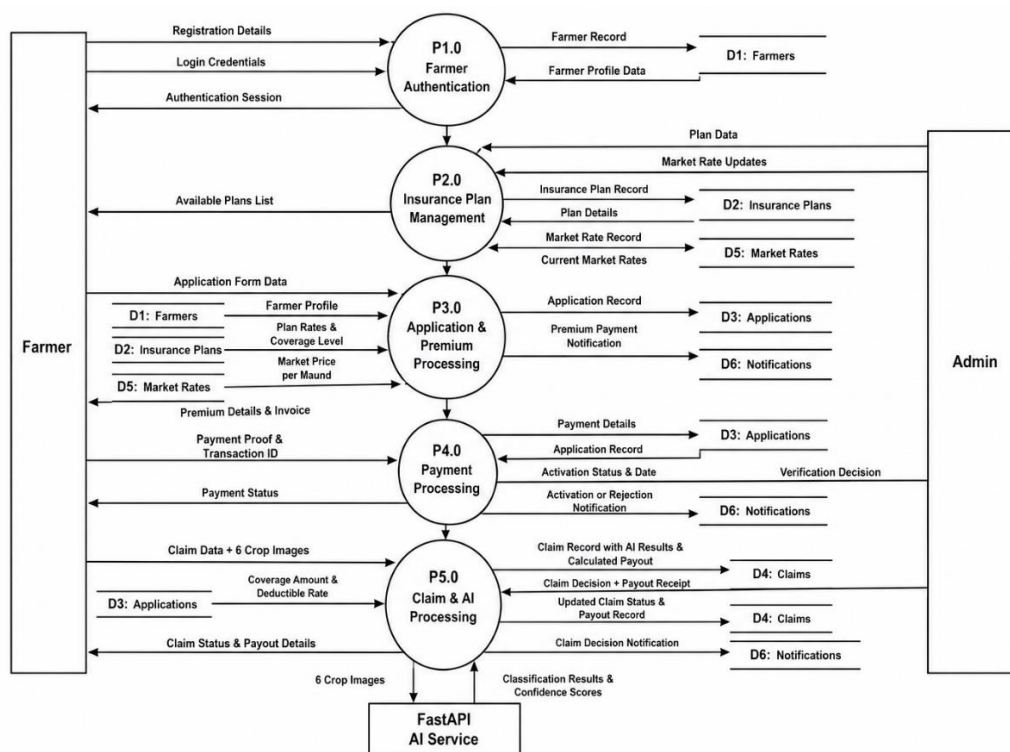


Figure 2.14 Level 0 DFD

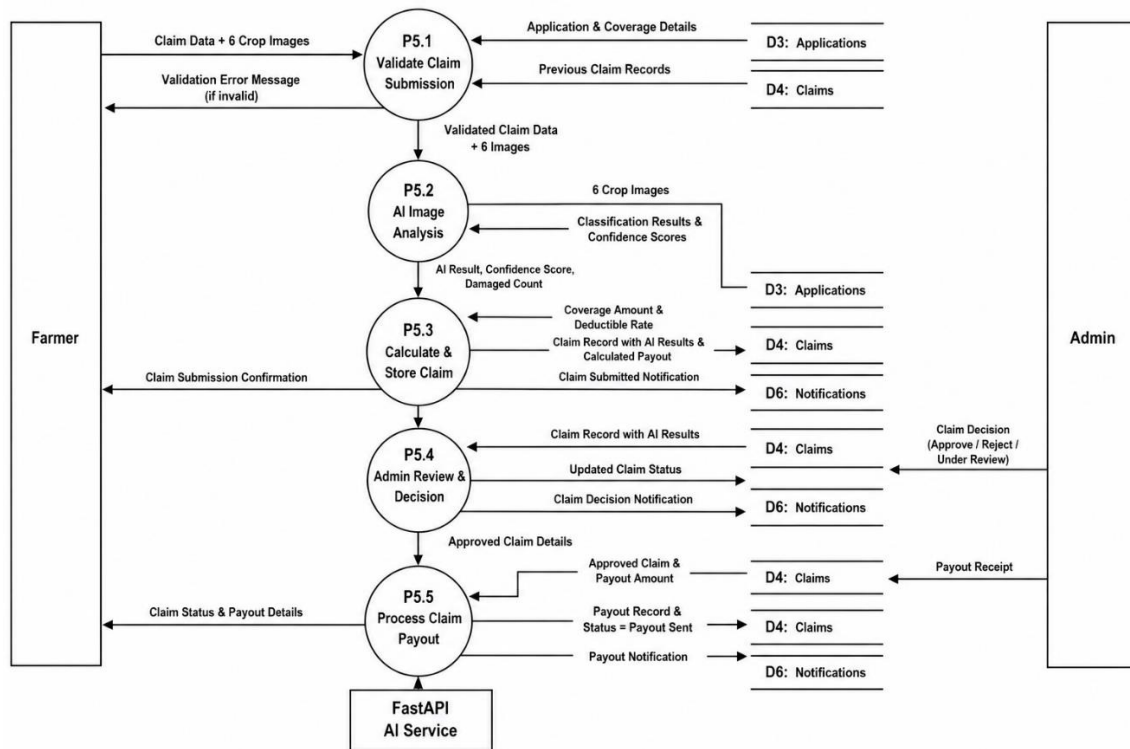
Level 1:

Figure 2.15 Level 1 DFD

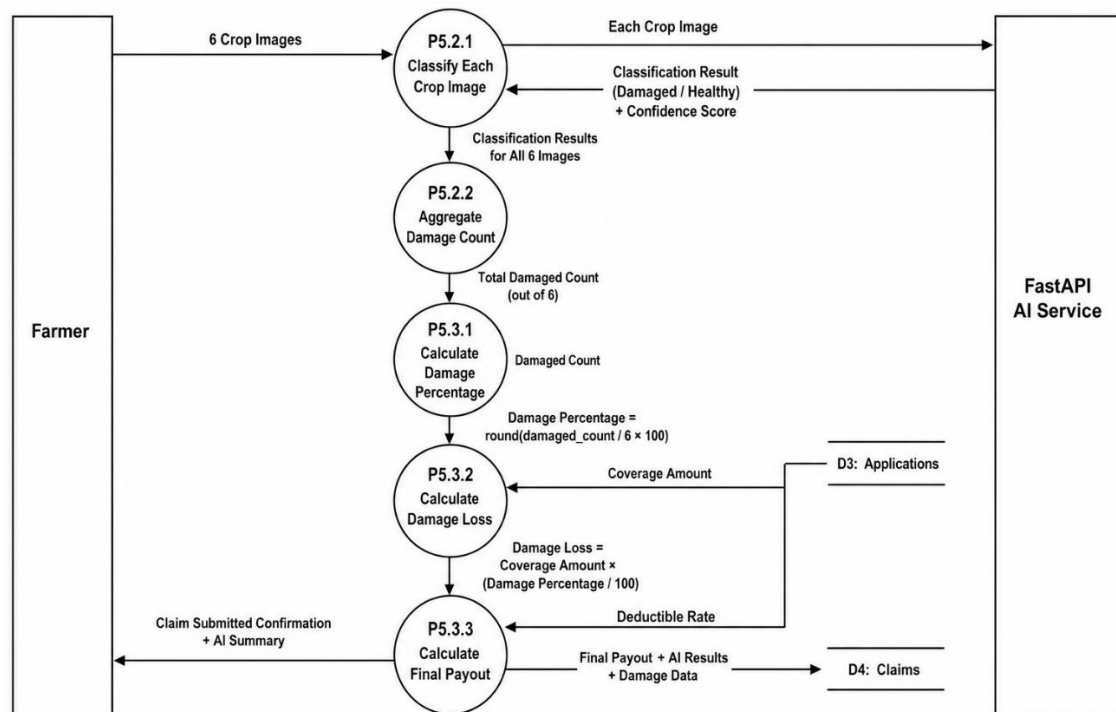
Level 2:

Figure 2.16 Level 2 DFD

Data Dictionary:

Table 2.11 presents the data dictionary for KissanSure, following the standard Yourdon-DeMarco data dictionary format [8].

Table 2.11: Data Dictionary

Data Element	Description	Type	Source	Destination
Registration Details	Farmer name, CNIC, phone, email, password, land area	Text/ Numeric	Farmer	D1: Farmers
Login Credentials	Email and password for authentication	Text	Farmer	P1.0
Authentication Session	Session established after successful login	Session	P1.0	Farmer
Insurance Plan Data	Plan name, coverage level, premium rate, deductible rate	Text/ Decimal	Admin	D2: Insurance Plans
Market Rate Updates	Current price per maund for Wheat or Rice	Decimal (PKR)	Admin	D5: Market Rates
Application Form Data	Crop type, insured area, expected yield, irrigation type, bank account	Text/ Numeric	Farmer	D3: Applications
Premium Details	Calculated sum insured, coverage amount, final premium	Decimal (PKR)	P3.0	Farmer
Payment Proof	Scanned payment receipt and bank transaction ID	File/Text	Farmer	D3: Applications
Verification Decision	Admin approval or rejection of payment	Text	Admin	D3: Applications
Activation Status	Plan status updated to Active with activation date	Text/Date	P4.0	D3: Applications
Claim Data	Loss date, reason, damaged area, estimated loss	Text/ Numeric/ Date	Farmer	D4: Claims
Crop Images	6 evidence images of damaged crop (JPG/PNG)	Image File	Farmer	FastAPI AI Service
Classification Results	Damaged or Healthy result with confidence score per image	Text/ Decimal	FastAPI AI Service	D4: Claims
Damage Percentage	$\text{round}(\text{damaged_count} / 6 \times 100)$	Integer (%)	P5.3	D4: Claims
Final Payout	$\text{damage_loss} \times (1 - \text{deductible_rate} / 100)$	Decimal (PKR)	P5.3	D4: Claims
Claim Decision	Admin decision (Approved, Rejected, or Under Review)	Text	Admin	D4: Claims
Payout Receipt	Uploaded proof of payout issued to farmer	File	Admin	D4: Claims
Notifications	System alerts sent to farmer at key workflow events	Text	P3.0, P4.0, P5.0	D6: Notifications

2.3.6 ER Diagram:

An Entity-Relationship (ER) diagram provides a graphical representation of the entities within a system and the relationships among them [9]. It models the logical structure of the database by showing how data is organized and how entities are associated with each other. Figure 2.17 presents the ER diagram for KissanSure, comprising six entities:

- Farmer
- InsurancePlan
- Application
- Claim
- CropMarketRate
- Notification

A Farmer applies for one or many Applications under a selected InsurancePlan. Each Application can generate one or two Claims. A Farmer receives many Notifications. CropMarketRate stores the current market price per maund for each supported crop and is referenced during premium calculation.

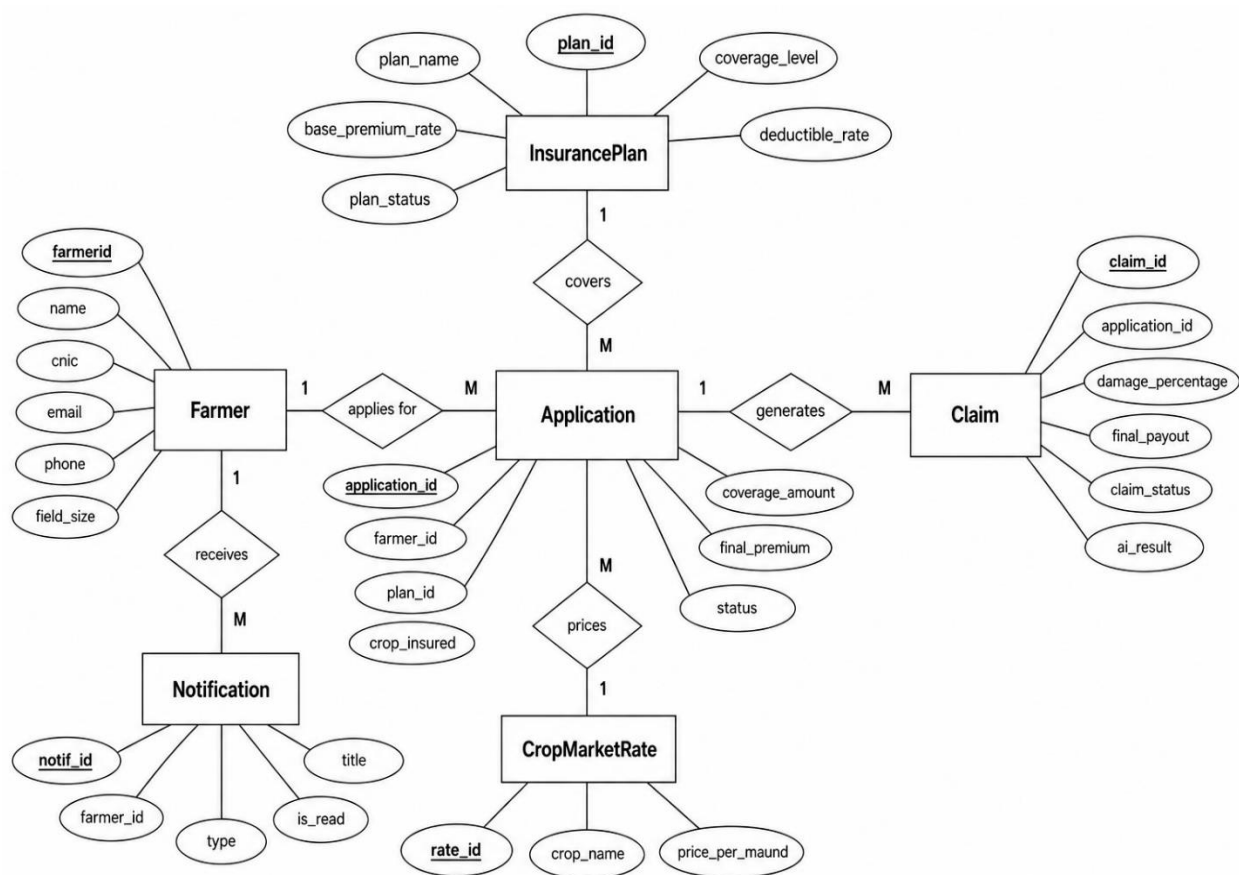


Figure 2.17 Entity Relationship Diagram

2.3.7 Database Model:

A database model shows the logical structure of a database, including the relationships and constraints that determine how data can be stored and accessed [10]. Figure 2.18 presents the database model for KissanSure, illustrating the six relational database tables:

- register_farmer
- insurance_plan
- farmer_applications
- insurance_claims
- crop_market_rates
- notifications

along with their column names, data types, primary keys, foreign keys, and the relationships between them.

The farmer_applications table serves as the central table, referencing both register_farmer and insurance_plan through foreign keys. The insurance_claims table references farmer_applications, and the notifications table references register_farmer.

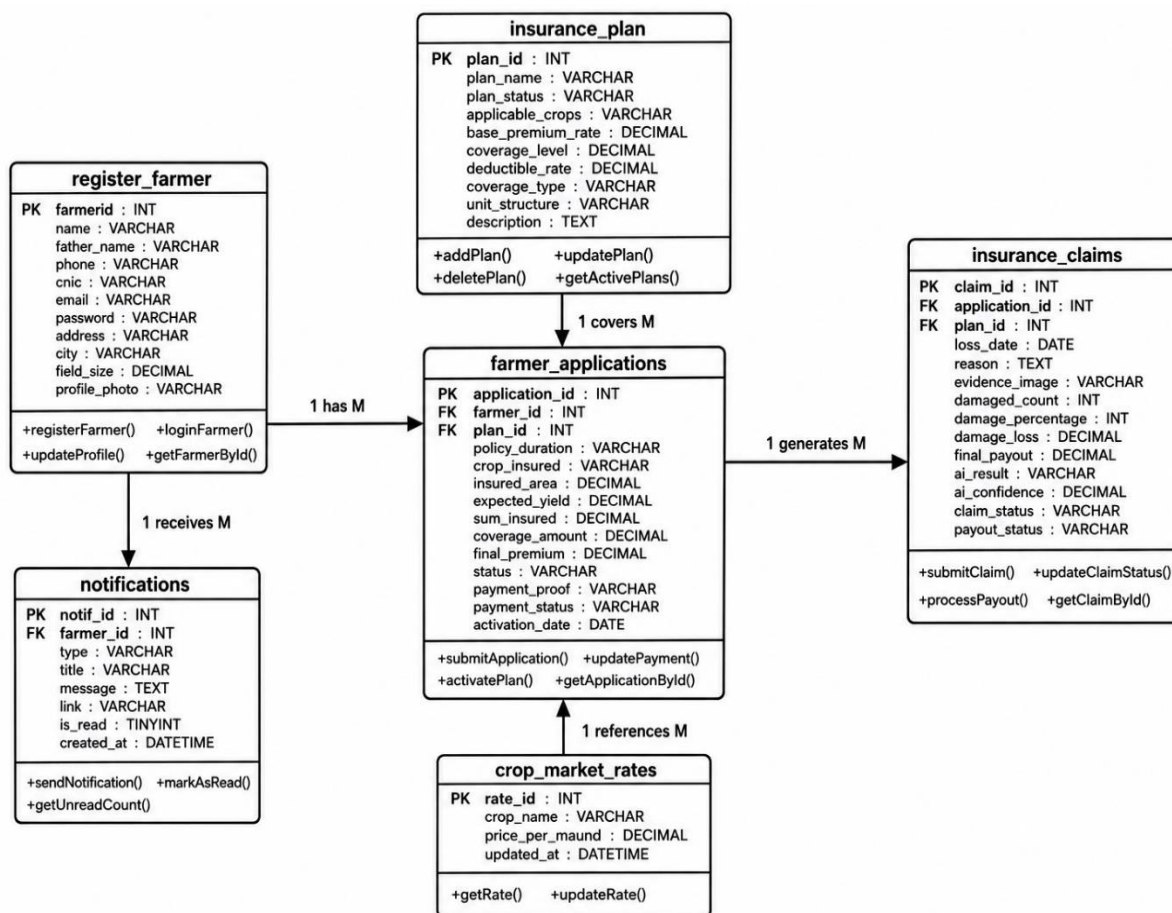


Figure 2.18 Database Model

2.3.8 Architecture:

4-Tier:

KissanSure follows a 4-tier (N-tier) architecture, extending the standard 3-tier model with a dedicated AI service layer that handles crop damage analysis independently from the main application logic. This separation allows the AI component to be updated or scaled without affecting the rest of the system.

Presentation Tier: Farmer and admin interfaces built with Bootstrap 5.3.3, HTML5, CSS, and JavaScript. Farmers can register, apply for insurance, submit payments, and file claims through a web browser. The admin panel supports plan management, payment verification, and claim processing.

Application Tier: Business logic layer built on PHP 8.2 running on Apache (XAMPP). Handles farmer authentication, premium calculation, payment processing, and claim management. Communicates with the database via MySQLi and forwards images to the AI tier via PHP cURL.

Data Tier: The data storage layer uses MySQL, accessed through PHP MySQLi. It stores all persistent data across six tables: register_farmer, insurance_plan, farmer_applications, insurance_claims, crop_market_rates, and notifications.

AI Service Tier: A standalone FastAPI microservice (Python/Uvicorn) running at <http://127.0.0.1:8000/analyze>. Hosts two MobileNetV2 binary classifiers (wheat and rice) built with PyTorch. The browser sends six crop images to this service via JavaScript Fetch API; the service returns a class (Damaged/Healthy) and confidence score per image, which are stored as hidden form fields and submitted with the claim.

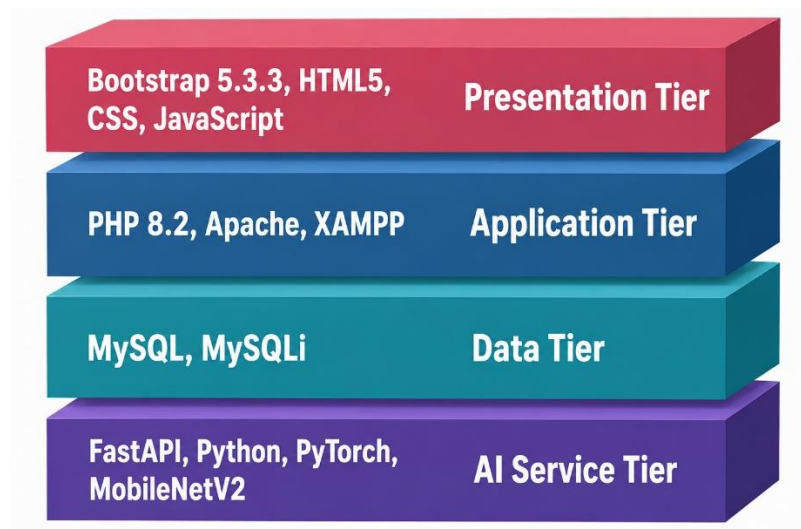


Figure 2.19 Application Architecture

Chapter 3 - RESULTS & DISCUSSION

3.1 Testing:

Software testing is the process of evaluating a software application to determine whether it satisfies the specified requirements and to identify defects before deployment. The objective of testing is to ensure that the system performs its intended functions correctly, handles invalid inputs gracefully, and maintains data integrity throughout all workflows.

For KissanSure, **black-box testing** (also known as functional testing) was adopted as the primary testing methodology. In black-box testing, the system is tested purely from the user's perspective based on inputs and expected outputs, without knowledge of the internal code structure. Test cases were designed directly from the functional requirements and the nine use cases documented in Chapter 2.

Testing was performed across the following areas of the system:

- Farmer registration, login, and password recovery
- Insurance plan browsing and application submission with automated premium calculation
- Premium payment proof submission and admin verification workflow
- Damage claim filing with six-image evidence upload and SHA-256 duplicate detection
- AI-powered crop damage analysis using MobileNetV2 (wheat and rice models)
- Automated payout calculation based on AI results and plan parameters
- Admin claim management including approval, rejection, and payout issuance
- Farmer notification system and real-time status updates

3.2 Test Cases:

Table 3.1: Farmer Registration

Test Case ID:	TC-01
Test Case Title:	Verify farmer self-registration with valid data
Test Case Priority:	High
Requirement:	FR01
Test Description:	Verify that a new user can successfully create a farmer account by providing all required personal, contact, and farm details.
Test Date:	05/31/2026
Pre-Conditions:	1. KissanSure application is running on XAMPP. 2. User does not have an existing account with the same phone or CNIC. 3. Database is accessible.
Dependencies:	Internet availability, XAMPP running
Test Steps:	1. Navigate to /user/farmer_register.php. 2. Enter full name, father's name, phone number (11 digits), CNIC (13 digits), email, address, city, and total field size. 3. Enter a password meeting complexity requirement (min 8 chars, 1 uppercase, 1 digit, 1 special character) and confirm it. 4. Observe real-time password strength indicator.

	5. Click the Register button.
Test Data	Name: Ali Ahmad Father: Ahmad Khan Phone: 03001234567 CNIC: 3310012345671 Email: ali@gmail.com City: Faisalabad Field Size: 5 acres Password: Farmer@123
Expected Results:	1. Password strength indicator updates in real-time. 2. Farmer account created in database. 3. System redirects to login page with success message.
Actual Results:	As expected
Post Conditions:	1. New farmer account created. 2. Farmer redirected to login page.
Status: (Pass/Fail)	Pass
Other Comments:	Tested with duplicate phone: system correctly displayed "Phone number already registered."

Table 3.2: Farmer Login

Test Case ID:	TC-02
Test Case Title:	Verify farmer login with valid credentials
Test Case Priority:	High
Requirement:	FR02
Test Description:	Verify that a registered farmer can authenticate into the system using phone number and password.
Test Date:	05/31/2026
Pre-Conditions:	1. Farmer account exists in database (TC-01 executed). 2. Application is running.
Dependencies:	TC-01 (Farmer Registration)
Test Steps:	1. Navigate to /user/login.php. 2. Enter registered phone number and correct password. 3. Click Login button.
Test Data	Phone: 03001234567 Password: Farmer@123
Expected Results:	1. PHP session created for authenticated farmer. 2. System redirects to home page. 3. Farmer's name displayed in navigation bar.
Actual Results:	As expected
Post Conditions:	1. Farmer is logged in with active session.
Status: (Pass/Fail)	Pass
Other Comments:	Tested with wrong password: system correctly displayed "Invalid phone number or password."

Table 3.3: Buy Insurance Plan

Test Case ID:	TC-03
Test Case Title:	Verify insurance plan application with automated premium calculation

Test Case Priority:	High
Requirement:	FR04, FR05
Test Description:	Verify that an authenticated farmer can browse available plans, submit an application, and receive a correctly calculated premium invoice.
Test Date:	05/31/2026
Pre-Conditions:	1. Farmer is logged in (TC-02 executed). 2. At least one insurance plan exists. 3. Crop market rate for Wheat configured by admin (PKR 3,500/Maund).
Dependencies:	TC-02 (Farmer Login), Standard insurance plan active in system by admin
Test Steps:	1. Navigate to Insurance Plans page. 2. Click "Apply Now" on the standard plan. 3. Select policy duration: Kharif. 4. Enter district: Faisalabad, bank account, insured area: 3 acres, irrigation type: Canal. 5. Select crop: Wheat, sowing date, expected yield: 50 Maunds, historical yield, seed variety. 6. Click Submit. 7. View the generated invoice.
Test Data	Plan: Standard (Coverage 75%, Premium Rate 10%, Deductible 25%) Crop: Wheat Expected Yield: 50 Maunds Market Rate: PKR 3,500/Maund, Insured Area: 3 acres
Expected Results:	1. Sum Insured = $50 \times 3,500 = \text{PKR } 175,000$. 2. Coverage Amount = $175,000 \times 75\% = \text{PKR } 131,250$. 3. Final Premium = $131,250 \times 10\% = \text{PKR } 13,125$. 4. Application saved with "Pending Payment" status. 5. Farmer redirected to invoice page with correct breakdown.
Actual Results:	As expected
Post Conditions:	1. Insurance application created with Pending Payment status. 2. Invoice displays correct calculated values.
Status: (Pass/Fail)	Pass
Other Comments:	Tested with insured area exceeding total field size: system correctly displayed validation error.

Table 3.4: Submit Payment Proof

Test Case ID:	TC-04
Test Case Title:	Verify premium payment proof submission
Test Case Priority:	High
Requirement:	FR06
Test Description:	Verify that a farmer with a pending payment application can upload payment proof and transaction ID for admin verification
Test Date:	05/31/2026
Pre-Conditions:	1. Farmer is logged in. 2. Farmer has at least one application with "Pending Payment" status (TC-03 executed).
Dependencies:	TC-03 (Buy Insurance Plan)

Test Steps:	1. Navigate to My Applications page. 2. Click "Submit Payment" on the Pending Payment application. 3. Upload a valid payment screenshot (JPG/PNG/WEBP/PDF, ≤5 MB). 4. Enter bank transaction reference number. 5. Select payment date. 6. Click Submit.
Test Data	Payment File: payment_receipt.jpg (valid JPG ≤5 MB) Transaction ID: TXN20260531001 Payment Date: 05/31/2026
Expected Results:	1. Payment proof stored on server. 2. Application status changes to "Payment Submitted". 3. Submission visible to admin on applications dashboard.
Actual Results:	As expected
Post Conditions:	1. Application status is "Payment Submitted". 2. Payment proof is stored.
Status: (Pass/Fail)	Pass
Other Comments:	Tested with file exceeding 5 MB: system correctly displayed file size validation error.

Table 3.5: Admin Verify Payment & Activate Plan

Test Case ID:	TC-05
Test Case Title:	Verify admin payment verification and policy activation
Test Case Priority:	High
Requirement:	FR14
Test Description:	Verify that the admin can review an uploaded payment proof and activate the farmer's insurance policy
Test Date:	05/31/2026
Pre-Conditions:	1. Admin is logged in (/admin/login.php). 2. At least one application with "Payment Submitted" status exists (TC-04 executed).
Dependencies:	TC-04 (Submit Payment Proof)
Test Steps:	1. Admin navigates to View Applications page. 2. Locates application with "Payment Submitted" status. 3. Click "Verify Payment" to open verification page. 4. Reviews application details and uploaded payment proof. 5. Clicks "Verify Payment & Activate Policy". 6. Confirms in the confirmation modal.
Test Data	Application with Payment Submitted status and valid uploaded payment proof
Expected Results:	1. Application status changes to "Active". 2. Activation date recorded in database. 3. Farmer receives in-system notification: "Insurance Plan Activated!".
Actual Results:	As expected
Post Conditions:	1. Policy is Active. 2. Activation date is set. 3. Farmer is notified.
Status: (Pass/Fail)	Pass
Other Comments:	Tested rejection: status correctly reverted to "Pending Payment" and farmer received rejection notification with reason.

Table 3.6: Submit Damage Claim

Test Case ID:	TC-06
Test Case Title:	Verify damage claim submission with six evidence images
Test Case Priority:	High
Requirement:	FR08
Test Description:	Verify that a farmer with an active policy can file a crop damage claim with loss details and exactly six photographic evidence images
Test Date:	05/31/2026
Pre-Conditions:	1. Farmer is logged in. 2. Farmer has at least one Active insurance policy (TC-05 executed). 3. Farmer has not reached the claim limit for the selected policy. 4. FastAPI AI microservice is running on http://127.0.0.1:8000.
Dependencies:	TC-05 (Admin Verify Payment & Activate Plan), FastAPI server running
Test Steps:	1. Navigate to "Apply for Claim" page. 2. Select the active policy from the dropdown. 3. Enter loss date, damaged area (2 acres), estimated loss (PKR 20,000), and loss reason (Disease Outbreak). 4. Upload exactly 6 different crop evidence images. 5. Click "Run AI Analysis" and wait for results. 6. Review per-image verdicts and damage summary. 7. Click Submit.
Test Data	Policy: Active Wheat policy Loss Date: 05/31/2026 Damaged Area: 2 acres Estimated Loss: PKR 20,000 Reason: Disease Outbreak 6 unique wheat crop images (at least 1 damaged)
Expected Results:	1. AI analysis completes showing per-image verdicts with confidence percentages. 2. Damage percentage and estimated payout calculated and displayed. 3. Claim record saved with AI Analyzed status. 4. Farmer redirected to My Claims page.
Actual Results:	As expected
Post Conditions:	1. Claim saved with AI Analyzed status. 2. Damage percentage and payout stored correctly.
Status: (Pass/Fail)	Pass
Other Comments:	Tested duplicate image detection: system correctly displayed error and rejected the duplicate before submission.

Table 3.7: AI Damage Analysis

Test Case ID:	TC-07
Test Case Title:	Verify AI crop damage classification and payout calculation
Test Case Priority:	High
Requirement:	FR09
Test Description:	Verify that the MobileNetV2 AI model correctly classifies uploaded images and that damage percentage and final payout are accurately calculated
Test Date:	05/31/2026

Pre-Conditions:	1. FastAPI AI microservice running on http://127.0.0.1:8000/analyze. 2. Wheat and Rice MobileNetV2 models (.pth files) loaded. 3. Farmer is on claim submission page with 6 images ready.
Dependencies:	FastAPI server running with wheat and rice models loaded
Test Steps:	1. Upload 6 wheat crop images (4 visibly damaged, 2 healthy) on the claim form. 2. Ensure crop type is set to Wheat. 3. Click "Run AI Analysis". 4. Observe per-image classification results. 5. Observe overall damage summary and payout calculation.
Test Data	6 wheat images: 4 damaged, 2 healthy Active Standard plan: Coverage Amount = PKR 131,250, Deductible Rate = 25%
Expected Results:	1. Each image returns verdict: Damaged or Healthy with confidence percentage. 2. Damaged Count = 4, Damage Percentage = $(4/6) \times 100 = 67\%$. 3. Damage Loss = $131,250 \times 0.67 = \text{PKR } 87,937.50$. 4. Final Payout = $87,937.50 \times (1 - 0.25) = \text{PKR } 65,953.13$. 5. Submit button enabled.
Actual Results:	As expected
Post Conditions:	1. AI results stored in database with claim record. 2. Payout figures are correct.
Status: (Pass/Fail)	Pass
Other Comments:	Tested with all 6 healthy images: damage % correctly showed 0%, damage loss and final payout calculated as PKR 0. Submit button remained enabled, farmer can still submit and admin reviews the claim for final decision.

Table 3.8: Admin Review & Decide Claim

Test Case ID:	TC-08
Test Case Title:	Verify admin claim approval and farmer notification
Test Case Priority:	High
Requirement:	FR15
Test Description:	Verify that the admin can review an AI-analyzed claim with the damage report and approve it with the farmer receiving a notification
Test Date:	05/31/2026
Pre-Conditions:	1. Admin is logged in. 2. At least one claim with "AI Analyzed" status exists (TC-06 executed).
Dependencies:	TC-06 (Submit Damage Claim)
Test Steps:	1. Admin navigates to View Claims page. 2. Locates claim with "AI Analyzed" status. 3. Click "View AI Report" to review per-image verdicts and damage assessment. 4. Returns to claims list and clicks "Approve" on the claim. 5. Confirms approval in the confirmation modal.
Test Data	Claim with AI Analyzed status AI report showing 4/6 images damaged, damage percentage 67% Final payout PKR 65,953.13
Expected Results:	1. Claim status changes to "Approved". 2. Farmer receives in-system notification: "Claim Approved." 3. Payout issuance option available for this claim in admin panel.
Actual Results:	As expected
Post Conditions:	1. Claim status is Approved.

	2. Farmer is notified. 3. Admin can proceed to issue payout.
Status: (Pass/Fail)	Pass
Other Comments:	Tested "Mark as Under Review": claim status correctly changed to "Under Review" and farmer was notified accordingly.

Table 3.9: Process Claim Payout

Test Case ID:	TC-09
Test Case Title:	Verify payout issuance and farmer payout receipt access
Test Case Priority:	High
Requirement:	FR15, FR16
Test Description:	Verify that the admin can issue a claim payout by uploading a receipt and the farmer can view and download it from their claims page
Test Date:	05/31/2026
Pre-Conditions:	1. Admin is logged in. 2. At least one claim is in "Approved" status (TC-08 executed).
Dependencies:	TC-08 (Admin Review & Decide Claim)
Test Steps:	1. Admin navigates to the payout issuance page for the approved claim. 2. Reviews payout calculation breakdown (Coverage Amount PKR 131,250, Damage % 67%, Deductible 25%, Final Payout PKR 65,953.13). 3. Uploads a payout receipt image (proof of bank transfer). 4. Clicks "Mark as Payout Sent" to confirm issuance. 5. Farmer logs in and navigates to My Claims page. 6. Farmer clicks "View Payout Receipt" on the claimed policy.
Test Data	Approved claim with Final Payout = PKR 65,953.13. Payout receipt: payout_receipt.jpg (valid JPG)
Expected Results:	1. Claim payout status changes to "Sent". 2. Payout date recorded in database. 3. Payout receipt stored and accessible. 4. Farmer receives in-system notification: "Payout Issued." 5. Farmer can view and download payout receipt from My Claims page.
Actual Results:	As expected
Post Conditions:	1. Payout status is Sent. 2. Farmer can access receipt. 3. Complete insurance lifecycle verified end-to-end.
Status: (Pass/Fail)	Pass
Other Comments:	None

3.3 Conclusion:

KissanSure was successfully designed, developed, and tested as a fully functional AI-integrated crop insurance system tailored to Pakistani farmers, covering the complete insurance lifecycle from registration to claim payout.

All nine test cases (TC-01 through TC-09), each corresponding to a system use case, returned a Pass status. The end-to-end sequence, registration, plan application, payment, AI analysis, claim approval, and payout, executed without errors, confirming correct integration across all modules.

The AI component, built on MobileNetV2 and served via a FastAPI microservice, is the system's most technically significant feature. The wheat model achieved ~95% validation accuracy and the rice model ~97%. During testing, the service correctly classified images, calculated damage percentages, and produced accurate payouts based on coverage and deductible parameters. Per-image confidence scores provide transparency for fair claim decisions.

The Bootstrap 5.3.3 interface delivers a responsive, consistent experience across devices. Color-coded status badges, a card-based layout, and a guided claim submission flow ensure usability for non-technical farmers. The real-time notification system alerts farmers at every key event, payment status, claim updates, and payout issuance, eliminating the need for manual status checks. Security measures include bcrypt password hashing, session-based authentication, input sanitization via *mysqli_real_escape_string()*, and integer casting of all ID parameters.

In summary, KissanSure demonstrates how web technologies and AI can address a real agricultural problem. Future enhancements include live server deployment, SMS notifications, additional crop models, and payment gateway integration.

Chapter 4 - USER MANUAL

4.1 Farmer Portal:

The Farmer Portal is the primary interface through which registered farmers interact with the KissanSure system. It is accessible through any modern web browser and is fully responsive across desktop and mobile devices.

4.1.1 Home Page:

The home page is the main landing page of KissanSure, accessible to all visitors including unauthenticated users. It introduces the platform and provides access to all major features through a structured layout with multiple sections.

Topbar and Navigation Bar

The topbar displays the KissanSure brand name as a clickable link that returns to the home page from any section. Below it sits the main navigation bar containing the following links:

- **Home:** Returns to the home page
- **Farmer Registration:** Navigates to the registration form (visible to unauthenticated users only)
- **Insurance Plans:** Navigates to the full insurance plans listing page
- **My Application:** Navigates to the farmer's application tracking page
- **Apply for Claim:** Navigates to the damage claim filing page
- **My Claims:** Navigates to the farmer's submitted claims page
- **Login / Register buttons:** Shown on the right side for unauthenticated visitors
- **Notification Bell:** Shown for logged-in farmers with an unread count badge, opens a dropdown of recent notifications
- **Profile Dropdown:** Shows the farmer's profile photo and name, contains links to My Profile, Payment History, and Logout

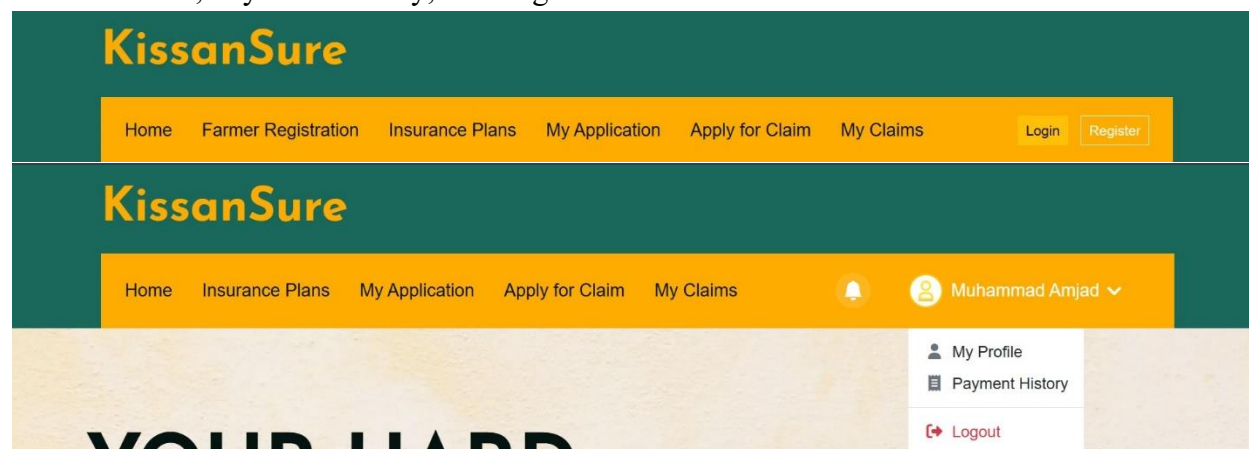


Figure 4.1 Navigation Bar

Hero Carousel

A sliding banner section with two slides, each displaying the platform's tagline and two action buttons:

- **Register Farm:** Redirects to the registration page (shown for unauthenticated visitors)
- **Login:** Redirects to the login page (shown for unauthenticated visitors)
- **Apply for Claim / View Plans:** Shown instead when the farmer is logged in

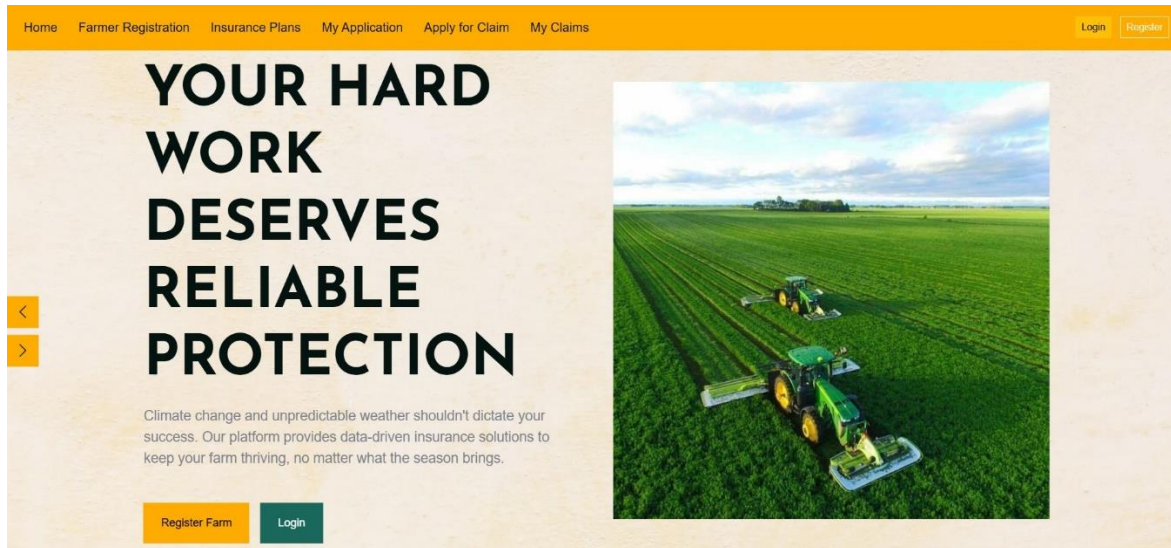


Figure 4.2 Hero Carousel

Contact Us Section

Contains a contact form for visitors to send queries directly to management. Fields include:

- **Your Name:** Text input for the visitor's full name
- **Your Email:** Email input
- **Your Phone:** Phone number input (format: 0300-1234567)
- **Your Query:** Textarea for the message
- **Send Message button:** Submits the form and sends an email notification to management via EmailJS
- **Contact on WhatsApp button:** Opens a WhatsApp chat link for direct contact

Figure 4.3 Contact Us Section

4.1.2 Farmer Registration:

The registration page allows new users to create a farmer account by providing personal, contact, and farm details. The system validates all inputs in real-time before account creation.

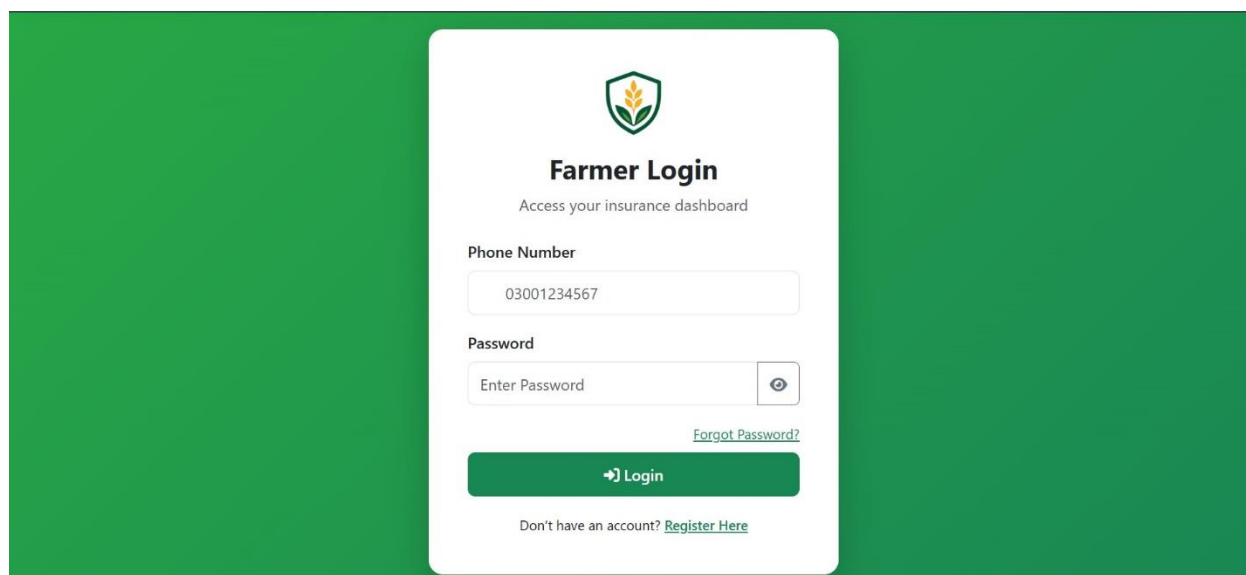
- **Full Name:** Text input for the farmer's complete name
- **Father's Name:** Text input for the farmer's father's name
- **Phone Number:** 11-digit numeric input; validated using regex
- **CNIC:** 13-digit numeric input; validated using regex
- **Email Address:** Optional email input
- **Address:** Text input for the farmer's residential address
- **City:** Text input for the farmer's city
- **Total Field Size (Acres):** Numeric input for the total farm area
- **Password:** Password input with a real-time strength indicator showing requirements (minimum 8 characters, 1 uppercase letter, 1 digit, 1 special character)
- **Confirm Password:** Must match the password field
- **Register button:** Submits the form; on success, redirects to the login page with a success message

Figure 4.4 Farmer Registration Page

4.1.3 Farmer Login:

The login page allows registered farmers to authenticate into the system using their phone number and password.

- **Phone Number:** 11-digit input field used as the login identifier
- **Password:** Password input with an eye icon toggle to show or hide the entered password
- **Login button:** Authenticates the farmer using bcrypt password verification; creates a PHP session on success and redirects to the home page
- **Forgot Password?:** Navigates to the password recovery page



The image shows a 'Farmer Login' page with a green background. At the top center is a logo of a shield with a green leaf and a yellow sun. Below the logo, the text 'Farmer Login' is displayed in bold, followed by 'Access your insurance dashboard'. There are two input fields: 'Phone Number' with the value '03001234567' and 'Password' with the placeholder 'Enter Password'. A 'Forgot Password?' link is below the password field. A green 'Login' button with a right arrow icon is at the bottom. Below the button is a link 'Don't have an account? Register Here'.

Figure 4.5 Farmer Login

4.1.4 Password Recovery:

Password recovery is a two-step process handled across two pages.

Step 1: Forgot Password Page

The farmer verifies their identity before being allowed to reset their password.

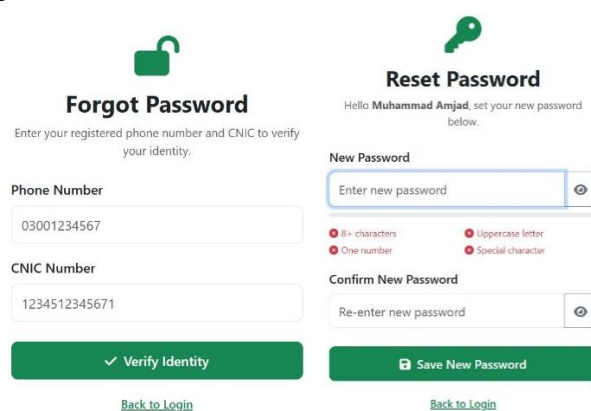
- **Phone Number:** Input field for the registered phone number
- **CNIC:** Input field for the registered CNIC number
- **Verify Identity button:** Checks the phone and CNIC combination against the database; if matched, redirects to the reset password page

Step 2: Reset Password Page

After identity verification, the farmer sets a new password.

- **New Password:** Password input with real-time strength indicator enforcing the same complexity requirements as registration
- **Confirm New Password:** Must match the new password field

Reset Password button: Updates the password in the database with a new bcrypt hash and redirects to the login page



The image shows two side-by-side pages for password recovery. The left page is titled 'Forgot Password' with a green padlock icon. It asks the user to 'Enter your registered phone number and CNIC to verify your identity.' It has two input fields: 'Phone Number' (03001234567) and 'CNIC Number' (1234512345671). A green 'Verify Identity' button is at the bottom. The right page is titled 'Reset Password' with a green key icon. It says 'Hello Muhammad Amjad, set your new password below.' It has two input fields: 'New Password' (placeholder 'Enter new password') and 'Confirm New Password' (placeholder 'Re-enter new password'). Below the 'New Password' field is a strength indicator with four red circles and labels: '8+ characters', 'One number', 'Uppercase letter', and 'Special character'. A green 'Save New Password' button is at the bottom. Both pages have a 'Back to Login' link at the bottom.

Figure 4.6 Password Recovery Pages

4.1.5 Insurance Plans:

The Insurance Plans page displays all active insurance plans available for application. It is accessible to all visitors including unauthenticated users.

Each plan is displayed as a card containing:

- **Plan Name:** Name of the insurance plan (e.g., Basic, Standard, Premium)
- **Applicable Crops:** Crop types covered under the plan
- **Base Premium Rate (%):** The rate used to calculate the final premium
- **Coverage Level (%):** The percentage of the sum insured that is covered
- **Deductible Rate (%):** The percentage deducted from the damage loss before payout
- **Coverage Type:** Type of coverage offered
- **Apply Now button:** Navigates to the insurance application form for the selected plan (requires login)

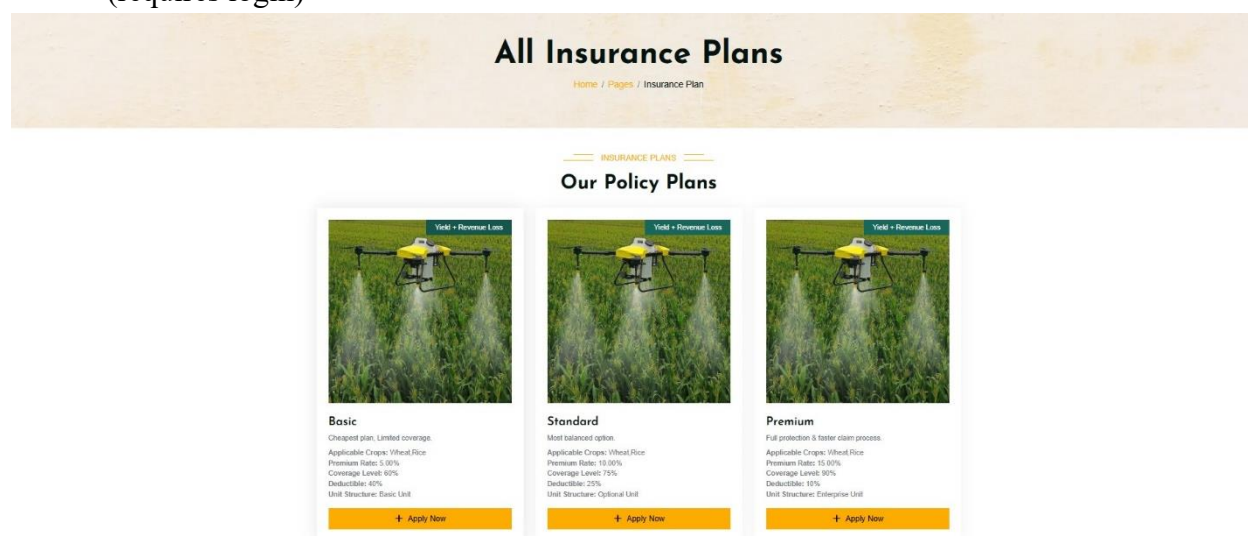


Figure 4.7 Insurance Plans Page

4.1.6 Insurance Application Form:

The application form allows an authenticated farmer to apply for a selected insurance plan by providing land and crop details. The form is pre-filled with the farmer's registered personal details. The form is divided into four sections:

Policy Duration

- **Kharif / Rabi radio buttons:** Selects the crop season for the policy

Farmer Details (read-only, auto-filled)

- **Personal Information:** Full Name, Father's Name, CNIC, Phone Number, pulled from the farmer's registered profile

Land Details

- **District:** Dropdown or text input for the district where the farm is located
- **Bank Account Number:** Input for the farmer's bank account for payout deposits
- **Insured Area (Acres):** Numeric input; must not exceed the farmer's total registered field size

- **Irrigation Type:** Dropdown to select irrigation method (Canal, Tubewell, Rainfed, etc.)

Crop Details

- **Crop Type:** Dropdown to select the crop (Wheat or Rice)
- **Sowing Date:** Date picker for the crop sowing date
- **Expected Yield (Maunds):** Numeric input for the expected harvest quantity
- **Historical Yield:** Input for past yield data
- **Seed Variety:** Text input for the seed type used
- **Certified Seed checkbox:** Indicates whether certified seed was used
- **Submit button:** Calculates the premium and creates the application with Pending Payment status; redirects to the invoice page

CROP INSURANCE APPLICATION

Apply for Standard

1. Selected Insurance Plan

Plan Details
Standard (Coverage: 75%)

Policy Duration
--Select Season--

2. Farmer Personal Details

• Your registered profile details are auto-filled and locked. Only District and Bank Account need to be entered.

CNIC Number
3310282406276

Mobile Number
03217651800

Full Name
Muhammad Anjad

Father Name
Sardar Ali

Complete Address
P-102, Street #1, Zunair Town, Marzi Pura, Faisalabad, Faisalabad

District / Tehsil
Faisalabad

Bank Account Number (For Payout)
Required for Claim Indemnity

3. Land & Crop Information

Area to be Insured (Acres/Kanals)
10
Registered land: 10 acres --- Already insured: 0.0 acres --- Remaining: 10.0 acres

Primary Crop Insured
--Select Crop--

Sowing Date
mm/dd/yyyy

Irrigation Type
--Select Irrigation Type--

4. Yield & Risk Factors

Total Expected Yield (Maunds)
e.g. 80 for 2 acres of wheat

Historical Yield Data (Last 3 Years)
e.g., 40, 45, 38

Seed Type/Variety
e.g., Sawera 24, Super Basmati

☐ Using Certified Seed Variety

Submit Application

Figure 4.8 Insurance Application Form

4.1.7 Premium Invoice:

The invoice page is displayed automatically after a successful application submission. It provides a full financial breakdown of the insurance policy.

- **Farmer and farm details:** Displays registered personal and land information
- **Plan details:** Shows the selected plan name, crop, coverage type, and policy duration

- **Sum Insured:** Expected Yield $\times$ Crop Market Rate (PKR)
- **Coverage Amount:** Sum Insured $\times$ Coverage Level %
- **Final Premium:** Coverage Amount $\times$ Base Premium Rate % (amount payable by farmer)
- **Guaranteed Production:** Minimum yield guaranteed under the policy
- **Submit Payment button:** Navigates to the payment proof submission page
- **Print / Download button:** Opens the browser print dialog to print or save the invoice as PDF

— MY APPLICATIONS —

Your Application Invoice

Application submitted successfully! Your premium has been calculated. Please review the invoice below and upload your payment proof to activate your policy.

Insurance Quote / Invoice
Pending Payment

Farmer Details:

Muhammad Amjad

CNIC: 3310282408276

Contact: 03217651800

Application ID:

#CROP-00006

Date: 31-May-2026

Coverage Summary

Selected Plan:	Standard
Crop & Season:	Rice (Kharif)
Insured Area:	5.00 Acres
Total Expected Yield:	300 Maunds
Guaranteed Production:	225.00 Maunds
75% of your total yield	

Financial Breakdown

Sum Insured (Total Yield $\times$ Market Rate)	PKR 1,200,000.00
Coverage Amount (75% of Sum Insured)	PKR 900,000.00
Premium Rate	10.00% of Coverage Amount

Net Premium Payable **PKR 90,000.00**

Payment Instructions: Transfer PKR 90,000.00 to any of the accounts below, then upload your payment proof.

- Bank Account: 0123-4567890-001 (HBL — KissanSure Insurance)
- JazzCash: 0301-1234567
- EasyPaisa: 0315-7654321

Upload Payment Proof

Download/Print Invoice

Download as Image

Figure 4.9 Premium Invoice Page

4.1.8 Submit Payment Proof:

This page allows the farmer to upload proof of their premium payment for admin verification. It is accessible from the My Applications page for any application with Pending Payment status.

- **Upload Payment Proof:** File upload field accepting JPG, PNG, WEBP, or PDF formats; maximum file size is 5 MB
- **Transaction ID:** Text input for the bank transfer reference number
- **Payment Date:** Date picker for the date of payment
- **Submit button:** Stores the payment proof and updates the application status to Payment Submitted

Figure 4.10 Submit Payment Proof Page

4.1.9 My Applications:

The My Applications page displays all insurance applications submitted by the logged-in farmer along with their current status. Each application is shown as a card with action buttons based on its status.

- **Application cards:** Each card shows plan name, crop type, policy duration, insured area, coverage amount, final premium, and current status badge
- **Status badges:**
 - **Pending Payment (yellow):** Payment not yet submitted
 - **Payment Submitted (blue):** Awaiting admin verification
 - **Active (green):** Policy is active
 - **Rejected (red):** Payment was rejected by admin
- **Submit Payment button:** Shown for Pending Payment applications; navigates to payment proof submission
- **View Invoice button:** Shown for Active applications; opens the premium invoice
- **Apply for Claim button:** Shown for Active applications; navigates to the claim filing page
- **Rejection reason notice:** Displayed when payment has been rejected by admin

MY APPLICATIONS

Your Insurance Applications

Muhammad Amjad

CNIC: 3310282408276

Plan: Standard

Crop: Rice

Area: 5.00

Season: Kharif

Status: Plan Not Activated - Premium Not Paid

Action:

View Invoice
Submit Payment

Applied On: 2026-05-31 18:00:06

Figure 4.11 My Applications Page

4.1.10 File a Damage Claim:

The claim filing page allows a farmer with an active policy to submit a crop damage claim. The page integrates AI analysis before final submission.

- **Select Policy dropdown:** Lists all active policies; selecting a policy auto-fills the crop type
- **Loss Date:** Date picker for when the damage occurred
- **Damaged Area (Acres):** Numeric input; must not exceed the insured area
- **Estimated Loss (PKR):** Farmer's own estimate of the financial loss
- **Loss Reason:** Dropdown to select the cause of damage (Right now only Disease Outbreak is available option)
- **Evidence Image Upload grid:** Six image upload slots for crop damage photographs; duplicate images are detected and rejected automatically using SHA-256 hashing; thumbnails are shown after upload
- **Run AI Analysis button:** Sends all six images to the FastAPI AI microservice; displays per-image results after analysis
- **Submit Claim button:** Enabled after AI analysis completes; saves the claim record to the database

CLAIM APPLICATION

Apply for Damaged Claim

Upload 6 images of your crop from different angles for AI analysis.

Submit Insurance Claim

Select Your Policy

Standard ▼

Crop Type

Rice ▼

Date of Loss

05/31/2026 📅

Damaged Area (Acres)

3

You insured 5 acres. Damaged area cannot exceed this.

Estimated Loss (PKR)

500000

For 3 damaged acres out of 5 insured acres, max realistic loss is PKR 540,000.

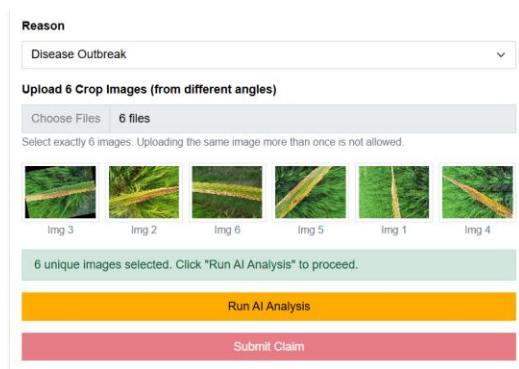


Figure 4.12 File a Damage Claim Page

4.1.11 AI Analysis Result:

After clicking Run AI Analysis, the results are displayed on the same claim filing page without a page reload. The AI analyses each of the six uploaded images using the crop-specific MobileNetV2 model.

- **Per-image verdict badges:** Each image slot displays a Damaged (red badge) or Healthy (green badge) label along with the confidence percentage
- **Overall damage summary bar:** Shows total damaged image count out of 6, calculated damage percentage, and damage severity label (Low / Moderate / Severe)
- **Decision message:** Informs the farmer that admin will review the claim application to give final decision
- **Submit Claim button:** Becomes active after analysis; farmer reviews results and clicks to finalize submission

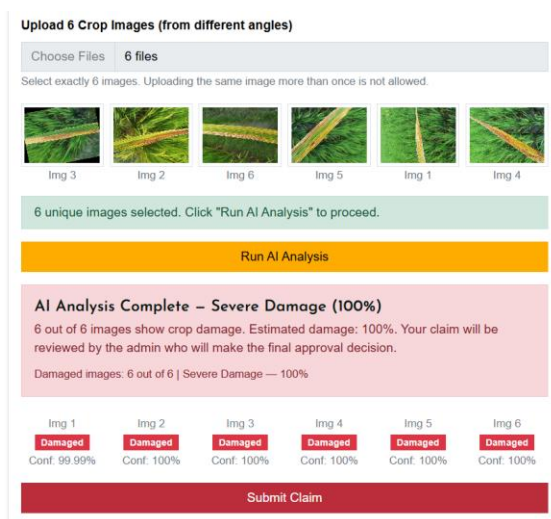


Figure 4.13 AI Analysis Results Display

4.1.12 My Claims:

The My Claims page displays all damage claims submitted by the logged-in farmer. Each claim is shown as a card with its current status.

- **Claim cards:** Each card shows the first evidence image, crop type, loss date, loss reason, estimated loss, AI damage level badge, and current claim status badge

- **Status badges:**
 - **AI Analyzed (blue):** Claim submitted and awaiting admin review
 - **Under Review (yellow):** Admin has flagged for manual inspection
 - **Approved (green):** Claim approved by admin
 - **Rejected (red):** Claim rejected by admin
 - **Payout Sent (green):** Compensation has been issued
- **View AI Report button:** Opens the detailed AI damage assessment report for the claim
- **View Payout Receipt button:** Appears when payout has been issued; opens the payout receipt page

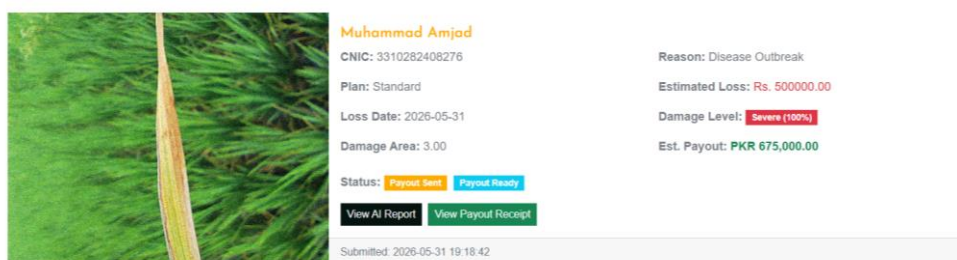
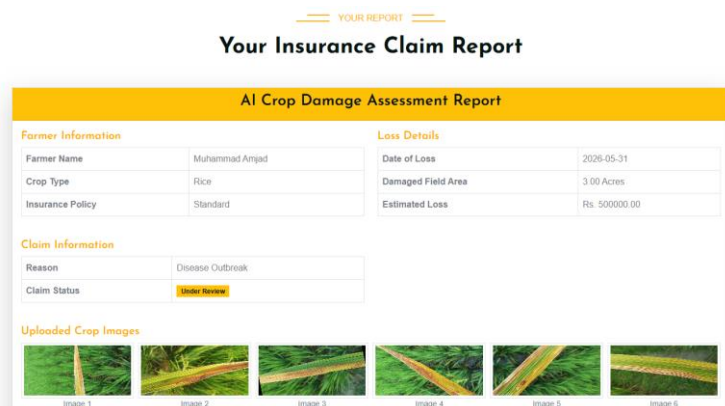


Figure 4.14 My Claims Page

4.1.13 AI Damage Report:

The AI Damage Report page provides a complete and printable breakdown of the AI assessment for a submitted claim.

- **Claim summary:** Shows policy details, loss date, reason, damaged area, and estimated loss
- **Evidence photographs:** All six uploaded crop images displayed in a grid
- **Per-image classification table:** A table showing image number, AI verdict (Damaged/Healthy), and confidence percentage for each image
- **Overall assessment:** Shows total damaged images, damage percentage, and eligibility verdict
- **Payout calculation breakdown:** Coverage Amount → Damage Loss → Deductible → Final Payout
- **Print/Download button:** Opens the browser print dialog for submission to financial institutions or admin



AI Analysis Result

AI Analysis Complete - **Severe Damage (100%)**
6 out of 6 submitted images show crop damage

Your claim is pending admin review. Compensation is calculated below

AI Image Classification	Damage Detected
Damaged Images	6 out of 6
Damage Detected	Severe Damage — 100%
Coverage Amount	PKR 900,000.00
Damage Loss (Coverage × 100%)	PKR 900,000.00
Deductible (25%)	- PKR 225,000.00
Estimated Payout	PKR 675,000.00
Admin Decision	Pending Admin Review

Generated by AI Crop Damage Detection System — KissanSure

[Print Report](#) [Download PDF](#) [Back](#)

Figure 4.15 AI Damage Report Page

4.1.14 Payout Receipt:

The Payout Receipt page is accessible when the admin has issued the claim payout. It shows the payout details and allows the farmer to download the receipt.

- **Payout status badge:** Shows Payout Sent in green
- **Payout calculation summary:** Displays Coverage Amount, Damage Percentage, Damage Loss, Deductible Rate, and Final Payout amount
- **Payout date:** The date on which the payout was issued by admin
- **Payout receipt image/PDF:** The bank transfer receipt uploaded by the admin
- **Download button:** Allows the farmer to download the receipt for their records

MY CLAIMS

Claim Payout Details

Payout Summary

Farmer: **Muhammad Amjad** Plan: **Standard**
CNIC: 3310262408276 Payout Date: 31 May 2026

Loss & Damage

Loss Reason: Disease Outbreak
Loss Date: 2026-05-31
AI Result: **Damaged** (100% severity)

Payout Calculation

Coverage Amount	PKR 900,000.00
AI Damage Percentage	100.00%
Damage Loss (Coverage × Damage%)	PKR 900,000.00
Deductible (25% your share)	- PKR 225,000.00
Final Payout Amount	PKR 675,000.00

Figure 4.16 Payout Receipt Page

4.1.15 Payment History:

The Payment History page provides a record of all premium payments made by the farmer across all insurance applications.

- **Payment records table:** Each row shows: Plan Name, Crop Type, Final Premium (PKR), Transaction ID, Payment Date, Payment Status badge, and a View Invoice link
- **Payment Status badges:**
 - **Pending** - Payment proof not yet submitted
 - **Submitted** - Awaiting admin verification
 - **Verified** - Payment confirmed and policy activated

- **Rejected** - Payment was rejected by admin
- **View Invoice link:** Opens the premium invoice for the corresponding application

MY PAYMENTS

Payment History

A record of all premium payments for your insurance applications.

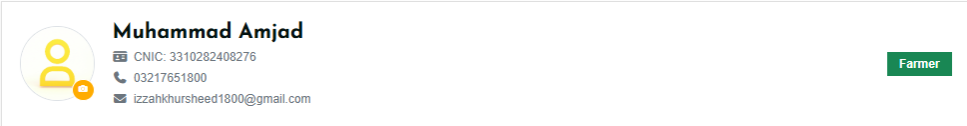
Plan	Crop	Premium	Transaction ID	Payment Date	Status	Action
Standard	Rice	PKR 90,000	TRXID0531202612	31 May 2026	Verified	Invoice

Figure 4.17 Payment History Page

4.1.16 Farmer Profile:

The Farmer Profile page displays the logged-in farmer's registered information and account statistics. It is accessible from the profile dropdown in the navigation bar.

- **Profile photo:** Displays the farmer's uploaded photo; a default avatar is shown if no photo has been uploaded
- **Upload/Change Photo button:** Opens a file upload dialog to upload or replace the profile photo
- **Personal Details card:** Shows Full Name, Father's Name, Phone Number, CNIC, and Email Address
- **Farm Details card:** Shows Address, City, and Total Field Size (Acres)
- **Account Statistics card:** Shows Total Applications, Total Claims, and the latest AI assessment result



1

Total Applications

1

Total Claims

✓

Policy Status: **Active**

Figure 4.18 Farmer Profile Page

4.1.17 Notifications:

The notification system keeps farmers informed of all key events related to their account. Notifications are accessible through the bell icon in the navigation bar on every page.

- **Bell icon:** Located in the top navigation bar; displays a red badge with the count of unread notifications
- **Notification dropdown:** Opens on clicking the bell icon; shows the latest 8 notifications in a scrollable panel
- **Notification items:** Each notification shows an icon (based on type), title, message, time elapsed since the event (e.g., "2 hours ago"), and a link to the relevant page
- **Unread indicator:** Unread notifications are highlighted in light blue with a small blue dot on the right

- **Mark all read link:** Appears at the top of the dropdown when unread notifications exist; marks all notifications as read and hides the badge

Notifications are triggered automatically by the system for the following events:

- Insurance policy activated
- Payment rejected by admin
- Claim status changed (Under Review, Approved, Rejected)
- Payout issued

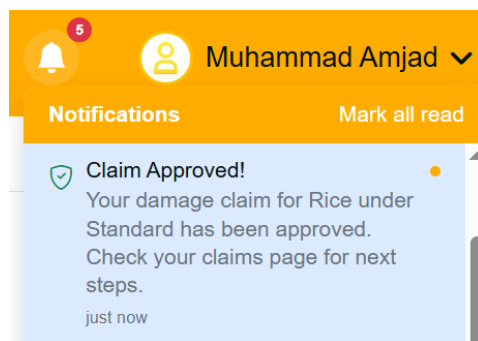


Figure 4.19 Notification Dropdown

References:

- [1] Government of Pakistan, Ministry of Finance. (2023). Pakistan Economic Survey 2022-23. Ministry of Finance, Islamabad. Retrieved from <https://www.finance.gov.pk>. This source provided statistical data on Pakistan's agricultural contribution to GDP cited in Chapter 1.
- [2] World Bank. (2023). Employment in Agriculture - Pakistan. World Bank Open Data. Retrieved from <https://data.worldbank.org/indicator/SL.AGR.EMPL.ZS?locations=PK>. This source provided Pakistan's agricultural employment percentage cited in Chapter 1.
- [3] Food and Agriculture Organization of the United Nations (FAO). (2022). Pakistan Floods 2022: Post-Disaster Needs Assessment. FAO, Rome. Retrieved from <https://data-in-emergencies.fao.org/datasets/pakistan-floods-post-disaster-needs-assessment-pdna-2022>. This report provided data on the extent of cropland damage during the 2022 Pakistan floods, including the 9.4 million acres of crop area inundated, cited in Chapter 1.
- [4] Food and Agriculture Organization of the United Nations (FAO). (2011). Agricultural Insurance in Asia and the Pacific Region. FAO Regional Office for Asia and the Pacific, Bangkok. Retrieved from <https://www.fao.org/4/i2344e/i2344e00.pdf>. This publication was referenced in Chapter 1 to provide context on large-scale crop insurance programs implemented in countries such as India (PM Fasal Bima Yojana) and China (Policy-based Agricultural Insurance).
- [5] Creately. (2024). *Class Diagram Relationships Explained with Examples*. Creately. Retrieved from <https://creately.com/guides/class-diagram-relationships/>. This source provided the notation guidelines and relationship types used in the class diagram in Section 2.3.4.
- [6] Visual Paradigm. (2024). *What is Data Flow Diagram (DFD)? How to Draw DFD?* Visual Paradigm Online. Retrieved from <https://www.visual-paradigm.com/guide/data-flow-diagram/what-is-data-flow-diagram/>. This source provided the theoretical foundation, notation standards, and levelling conventions for Data Flow Diagrams constructed in Section 2.3.5.
- [7] Visual Paradigm. (2024). *Context Diagram vs Level 0 DFD*. Visual Paradigm Skills. Retrieved from <https://skills.visual-paradigm.com/docs/common-dfd-mistakes-and-how-to-avoid-them/dfd-scope-mistakes/context-diagram-vs-level-0-dfd/>. This source clarified the distinction between the Context Diagram and Level 0 DFD used in the design of the DFD hierarchy in Section 2.3.5.
- [8] Yourdon, E. (1989). *Modern Structured Analysis*. Prentice-Hall, Englewood Cliffs, NJ. This textbook provided the standard Yourdon-DeMarco data dictionary format adopted for the Data Dictionary in Section 2.3.5.

- [9] SmartDraw. (2024). *Entity Relationship Diagram (ERD)*. SmartDraw. Retrieved from <https://www.smartdraw.com/entity-relationship-diagram/>. This source provided the theoretical foundation and notation guidelines for the Entity-Relationship Diagram constructed in Section 2.3.6.
- [10] Datanamic. (2024). *Introduction to Database Modeling*. Datanamic. Retrieved from <https://www.datanamic.com/support/lt-dez005-introduction-db-modeling.html>. This source provided the theoretical foundation for database modeling concepts used in Section 2.3.7.
- [11] Kaggle Inc. (2024). *Kaggle Datasets: Open Data Platform for Machine Learning and Data Science*. Kaggle. Retrieved from <https://www.kaggle.com/datasets>. Wheat and rice crop disease image datasets used for training the MobileNetV2 classification models in this project were sourced from this platform.
- [12] Mendeley Data. (2024). *Mendeley Data: Research Data Repository*. Elsevier. Retrieved from <https://data.mendeley.com>. Supplementary crop image datasets for wheat and rice disease classification were sourced from this platform.